Enhancing Provable Security and Efficiency of Permutation-based DRBGs

Woohyuk Chung[1], Seongha Hwang[1], Hwigyeom Kim[2]*, and Jooyoung Lee[1]**

KAIST, Daejeon, Korea {hephaistus,mathience98,hicalf}@kaist.ac.kr
Norma Inc., Seoul, Korea rlagnlrua4@gmail.com

Abstract. We revisit the security analysis of the permutation-based deterministic random bit generator (DRBG) discussed by Coretti et al. at CRYPTO 2019. Specifically, we prove that their construction, based on the sponge construction, and hence called Sponge-DRBG in this paper, is secure up to $O\left(\min\left\{2^{\frac{c}{2}}, 2^{\frac{\lambda}{2}}\right\}\right)$ queries in the seedless robustness model, where λ is the required min-entropy and c is the sponge capacity. This significantly improves the provable security bound from the existing $O\left(\min\left\{2^{\frac{c}{3}}, 2^{\frac{\lambda}{2}}\right\}\right)$ to the birthday bound. We also show that our bound is tight by giving matching attacks.
As the Multi-Extraction game-based reduction proposed by Chung et al. at Asiacrypt 2024 is not applicable to Sponge-DRBG in a straightforward manner, we further refine and generalize the proof technique so that it can be applied to a broader class of DRBGs to improve their provable security.
We also propose a new permutation-based DRBG, dubbed POSDRBG, with almost the optimal output rate 1, outperforming the output rate $\frac{r}{n}$ of Sponge-DRBG, where n is the output size of the underlying permutation and $r = n - c$. We prove that POSDRBG is tightly secure up to $O\left(\min\left\{2^{\frac{c}{2}}, 2^{\frac{\lambda}{2}}\right\}\right)$ queries. Thus, to the best of our knowledge, POSDRBG is the first permutation-based DRBG that achieves the optimal output rate of 1, while maintaining the same level of provable security as Sponge-DRBG in the seedless robustness model.

Keywords: Deterministic random bit generator, Seedless robustness model, Cryptographic sponge, Permutation, Provable security

1 Introduction

Deterministic Random Bit Generator. Random number generation is essential for ensuring the security of cryptographic operations such as generating cryptographic keys, initialization vectors (for symmetric encryption modes), and

* Hwigyeon Kim contributed to this work during his PhD study at KAIST.
** Corresponding author.

random errors (for lattice and code-based cryptography). A deterministic random bit generator (DRBG) is a deterministic algorithm that generates unpredictable random numbers from hardware entropy sources. A DRBG is used in various cryptographic protocols and systems, playing a crucial role in modern operating systems such as Linux's /dev/random [6,10] and Windows' Fortuna [13]. The security of DRBGs ensures the integrity and confidentiality of the cryptographic systems in use. In this paper, the focus is put on the provable security of DRBGs.

Structure of DRBGs and Their Output Efficiency. Following the formalism from [8,10,19] (with some simplification), a DRBG is defined as a pair of algorithms $\mathcal{G} = (\mathsf{refresh}, \mathsf{next})$, where

- $\mathsf{refresh}$ is an entropy accumulation algorithm that takes as input a state S and an input I (obtained from an entropy source), and returns a new state S' (with more entropy),
- next is a random bit generation algorithm that takes as input a state S and an output length len, and returns a new state S' and a binary string R of len bits.

So a DRBG accumulates entropy in its state S using $\mathsf{refresh}$ and then generates random bits from the state S via next. These algorithms are based on cryptographic primitives such as hash functions, block ciphers, and permutations.

In order to estimate the efficiency of a DRBG (focusing on the efficiency of the next algorithm), we define the *output rate* of a DRBG as $\mathsf{len}/(nB)$, where len is the number of output bits generated by next, n is the output size of the underlying primitive, and B is the number of primitive queries made by each query to next. So the notion of output rate captures how many output bits are used from the underlying primitive to generate a single random bit in the next algorithm.

Provable Security of DRBGs. The provable security of DRBGs is typically studied in terms of the notion of robustness security [1,7,10,11,15,16,17,18,19,21]. In the robustness model, an adversary is able to not only obtain the output of the DRBG but also leak or modify its internal state. The robustness security of a DRBG ensures that the output of the DRBG remains unpredictable to such an adversary. Recently, Hash-DRBG and HMAC-DRBG have been proved to be secure up to $O(\min\{2^{\frac{n}{2}}, 2^{\frac{\lambda}{2}}\})$ queries [21], while CTR-DRBG and OFB-DRBG are secure up to $O(\min\{2^{\frac{n}{2}}, 2^{\frac{\lambda}{3}}\})$ queries [7,16], where n is the output size of the underlying primitive and λ is the required min-entropy, respectively. However, their security proof is based on a rather unrealistic assumption that a DRBG is using a seed that is short, perfectly random, and independent of the entropy sources.

Seedless Robustness Model. It is unrealistic to assume that a DRBG operates independently of the hardware entropy sources. Coretti et al. [8] addressed this issue by introducing a seedless robustness model. In this model, they proved that Merkle-Damgård DRBG (denoted MD-DRBG in this paper) is secure up to

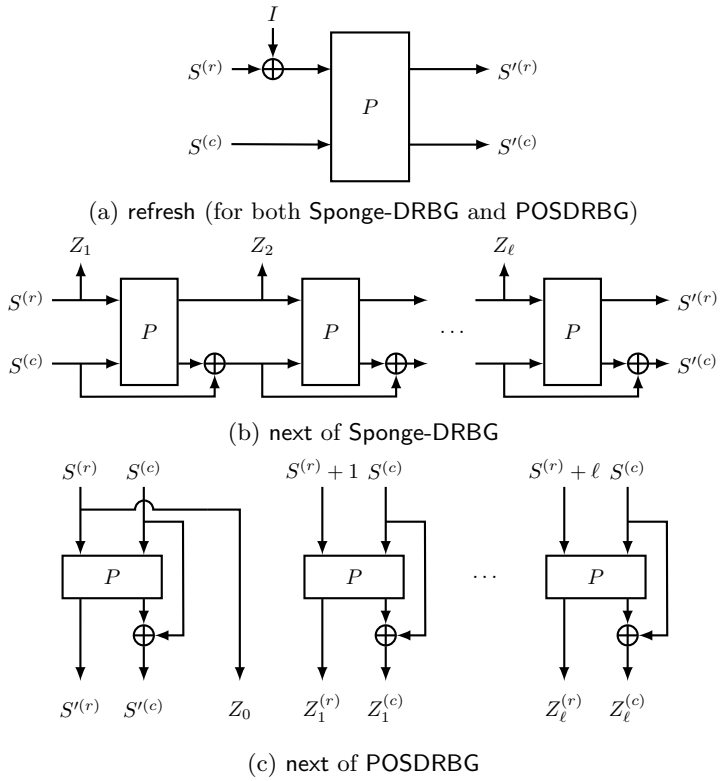
$O(\min\{2^{\frac{n}{3}}, 2^{\frac{\lambda}{2}}\})$ queries when the underlying primitive is modeled as a random oracle, and proved the same bound when the underlying compression function is replaced by the Davies-Meyer construction based on an ideal cipher (denoted MD-DM-DRBG). They also proved that Sponge DRBG (denoted Sponge-DRBG) is secure up to $O(\min\{2^{\frac{c}{3}}, 2^{\frac{\lambda}{2}}\})$ queries in the (public) random permutation model, where c is the capacity of the sponge construction.

On the other hand, they showed that CBC-based extractors are not secure in this model. Later, Hoang and Shen [16] showed that CTR-DRBG based on a CBC-based extractor is also insecure in the seedless model.

Permutation-based DRBG. A permutation-based DRBG uses an n-bit public permutation as its primitive, accumulating entropy in r bits of the permutation input and generating random bits from r bits of the permutation output, where r is the sponge rate. In practice, the Keccak (SHA-3) permutation [2,4,5,9,12] is often used as the underlying primitive, while it is modeled as a truly random permutation in the security proof of DRBGs based on it. The first permutation-based DRBG was proposed by Bertoni et al. [3]. Gaži and Tessaro subsequently modified its structure so that it is secure in the (seeded) robustness model [15]. Hutchinson [17] introduced a DRBG, named Reverie, with an enhanced output rate. Later, Coretti et al. [8] removed the seed component from Reverie, dubbed Sponge-DRBG, and proved its security in the seedless robustness model.

In the next algorithm, Sponge-DRBG extracts only r bits from the n-bit output of a single primitive query. In contrast, MD-DRBG and MD-DM-DRBG, based on hash functions and block ciphers, respectively, fully utilize the output of each primitive, achieving an output rate of 1. We conjecture that an output rate of 1 is optimal, as it ensures that the DRBG fully uses outputs of its underlying primitive. However, by design, permutation-based DRBGs fail to achieve this optimal rate. This raises a fundamental question: *whether a permutation-based DRBG can be designed to achieve an optimal output rate of* 1 *while maintaining security in the seedless robustness model.* Given the increasing adoption of permutation-based cryptographic designs, particularly in standards such as SHA-3 [12] and the lightweight cryptography contest winner Ascon [20], it would be relevant and interesting to address this efficiency gap. Motivated by this question, we propose POSDRBG, a novel construction that improves the output rate to 1 while preserving the security guarantees of Sponge-DRBG.

Security Proof Using Multi-Extractor Games. Dodis et al. [10] proved the robustness security of DRBGs by decomposing the robustness security game into recovering games and preserving games. Coretti et al. [8] modified this strategy by further decomposing the recovering game into an extraction game and a next game, and the preserving game into a maintaining game and a next game, respectively. The extraction game is defined for every REF query made with insufficient entropy, where the adversarial advantage against each game contains the term $p^2/2^n$, where p denotes the number of primitive queries. By applying the game-hopping argument, they obtained the bound containing $q_2p^2/2^n$, where q_2 is the maximum number of ROR queries. To prove the tight security of



(a) refresh (for both Sponge-DRBG and POSDRBG)

(b) next of Sponge-DRBG

(c) next of POSDRBG

Fig. 1: The structure of Sponge-DRBG and POSDRBG. $\oplus$ and $+$ denote bitwise XOR and modular addition, respectively.

Linux-DRBG, Chung et al. decomposed the seedless robustness game into three subgames, dubbed M-EXT, pREF, and pNEXT [6], respectively. The M-EXT game handles multiple extraction games simultaneously. In this way, they obtained the bound without q_2, leading to the tight bound with respect to the output size n of the underlying hash function of Linux-DRBG.

1.1 Contribuition

The contribution of this paper is summarized as follows (see also Table 1, 2, and Figure 1).

– **Tight security bound of Sponge-DRBG.** We revisit the security proof of Sponge-DRBG discussed in [8], where its security is proved below the birthday bound with respect to c in the seedless robustness model, while the tightness of the bound has been left open. In this paper, we prove that Sponge-DRBG is secure up to $O(\min\{2^{\frac{c}{2}}, 2^{\frac{\lambda}{2}}\})$. So the security bound is significantly improved

Table 1: Provable security (in bits) of DRBGs in the seedless robustness security model. λ is the min-entropy threshold of the entropy source, n is the output size of the underlying primitive, and r and c are the rate and the capacity of the sponge constructions, respectively, where $n = r + c$.

Scheme	Primitive	Output Rate	Old Bounds	Our Bounds
Sponge-DRBG	Ideal Permutation	$\frac{r}{n}$	$\min\{\frac{\lambda}{2}, \frac{c}{3}\}$[8]	$\min\{\frac{\lambda}{2}, \frac{c}{2}\}$
POSDRBG	Ideal Permutation	1	-	$\min\{\frac{\lambda}{2}, \frac{c}{2}\}$

up to the birthday bound from the existing bound $O(\min\{2^{\frac{c}{3}}, 2^{\frac{\lambda}{2}}\})$. We also show that the security bound is tight by giving matching attacks.

– **New security reduction.** Since the Multi-Extractor reduction used for the security proof of Linux-DRBG is not applicable to Sponge-DRBG and POSDRBG, we propose a new approach of decomposing the seedless robustness game into two separate games: M-Rec and M-Pres. This approach allows one to achieve a tight bound for Sponge-DRBG. Our reduction technique can be seen as a generalization of the Multi-Extractor reduction, which is applicable to a broader range of DRBGs including Sponge-DRBG, POSDRBG, and those proposed in [8].
– **POSDRBG: enhancing the output rate of Sponge-DRBG.** We propose a new permutation-based DRBG, dubbed POSDRBG (Parallel Output Sponge DRBG). Compared to the existing Sponge-DRBG [8,15], POSDRBG generates random numbers in parallel, enjoying an optimal output rate of 1 when len is sufficiently large (see Figure 1). We prove that POSDRBG is secure up to $O(\min\{2^{\frac{c}{2}}, 2^{\frac{\lambda}{2}}\})$ queries, and present matching attacks for POSDRBG to show the tightness of our bound. To the best of our knowledge, POSDRBG is the first permutation-based DRBG that achieves an optimal output rate of 1 with parallelizability while maintaining the same level of security as Sponge-DRBG.

1.2 More on POSDRBG: Design Rationale and Implementation

Design Rationale of POSDRBG. POSDRBG uses the same refresh algorithm as Sponge-DRBG. In Sponge-DRBG, each permutation query generates n bits, but only r bits are used as the DRBG output. This creates an inherent limitation, as all the bits produced by the permutation are not fully utilized. On the other hand, POSDRBG utilizes a counter in the upper r bits of the permutation input to generate multiple outputs in parallel. By applying feedforward to the lower c bits, POSDRBG can fully utilize the n-bit permutation outputs, achieving almost the optimal output rate 1 when len is sufficiently large.[1]

[1] In many real-world applications, a large amount of random bits should be efficiently generated. For example, 2^{37} random bits per second are needed for nonce gener-

Table 2: Time required to produce random bits (ms).

Scheme	Output length		
	50MB	100MB	200MB
Sponge-DRBG (sequential)	214.2	425.9	857.1
Sponge-DRBG (4-core)	110.5	220.8	441.7
POSDRBG (sequential)	148.8	298.2	595.4
POSDRBG (4-core)	72.3	143.8	288.7

Implementation. We compare the performance of Sponge-DRBG and POSDRBG, in particular, their next algorithms, when they are instantiated with the Keccak-$f[1600]$ permutation with $r = 1088$ and $c = 512$. Our experiments were done in an AMD Ryzen 7 2700X Eight-Core Processor (CPU@2.2GHz) using the XKCP library.[2]

Table 2 shows the time required for each DRBG to produce random bits of given lengths. In other words, we measured the time of next while ignoring the time for refresh. Due to the output rate, Sponge-DRBG requires n/r times as many permutation queries as POSDRBG to produce the same length of random output. Table 2 clearly shows that the difference in the number of permutation queries affects the actual time required to produce random bits of given length, where the $n/r = 1600/1088 \approx 1.4$ for our parameters:

- For sequential execution, POSDRBG is about 1.4 times faster than Sponge-DRBG due to their difference in output rate.
- For the 4-core setting, where we deploy four independent instances of Sponge-DRBG to each core, POSDRBG is about 1.4 times faster than Sponge-DRBG due to the output rate, too.

It is important to note that, in terms of entropy accumulation, each Sponge-DRBG instance operating on a separate core must perform independent refresh operations to prevent output collisions. In contrast, POSDRBG allocates a distinct counter for each core while maintaining a shared state, thereby reducing the computational overhead during the entropy accumulation phase.

Let L denote the maximum output block length produced by a single query to next, which can be set by the user. Due to the counter, POSDRBG requires additional $\log(L)$ bits of memory compared to Sponge-DRBG, where $\log(L)$ should be at most $\min\{r, \frac{c}{2}\}$. Note that $\log(L)$ can be appropriately chosen according to applications. For example, a 64 bit counter can be used on a Raspberry Pi and an 8 bit counter on an 8-bit microcontroller.

ation in Amazon's AES-GCM (see `https://csrc.nist.gov/Presentations/2023/practical-challenges-with-aes-gcm`) and 90TB of randomness is required by Differential Privacy algorithms [14].

[2] https://github.com/XKCP/XKCP

1.3 More on Our Proof Technique: M-Rec and M-Pres Games

In this section, we further elaborate on the limitations of the previous proof technique and our approach. We refer to Section 2.2 on the seedless robustness model and Appendix A on the detailed description of the M-EXT game to fully understand the technical details of this section. The M-EXT game, originally described in Linux-DRBG's syntax [6], is presented in the Appendix in the context of general DRBGs.

Limitation of Multi-Extractor Reduction. The M-EXT game in the Multi-Extractor reduction ensures that a DRBG can regain randomness even when its state is under the adversary's control, provided that the adversary generates entropy input I while satisfying the λ-legitimacy condition. To capture this, in the M-EXT game, the adversary selects the initial DRBG state S_0 and constructs the entropy input I accordingly. After executing M-EXT(S_0, I), the resulting state S is also given to the adversary, which is essential for the reduction proof used in Multi-Extractor reduction [6].

In the real world, M-EXT queries correspond solely to the refresh algorithm of the DRBG. Specifically, S is obtained by iteratively applying refresh using S_0 and I, whereas, in the ideal world, S is sampled uniformly at random. As shown in Figure 1, the refresh algorithm in permutation-based DRBGs uses a single query to the underlying permutation. Also, an adversary is allowed to make an inverse query to the permutation. Consequently, if $\lambda \leq r$, then the following $O(1)$ attack on the M-EXT game is possible against permutation-based DRBGs:

1. $\mathcal{A}_1$ selects $I \in \{0,1\}^r$, ensuring that I has min-entropy λ.
2. $\mathcal{A}_2$ chooses an initial state S_0 and shares it with $\mathcal{A}_1$ through S_{all}.
3. $\mathcal{A}_1$ queries M-EXT(S_0, I) and obtains the resulting state S, which is then passed to $\mathcal{A}_2$ via S_{all}.
4. $\mathcal{A}_2$ queries the inverse primitive oracle $\pi^{-1}(S)$, then obtains S_1.
5. $\mathcal{A}_2$ checks whether $S_0^{(c)} = S_1^{(c)}$. If true, it returns 1; otherwise, it returns 0.

In the real world, $S_1 = S_0 \oplus I_1 \parallel 0^c$ must hold, ensuring $S_0^{(c)} = S_1^{(c)}$. However, in the ideal world, this equality holds with probability close to $1/2^c$. Thus, the attack succeeds with a non-negligible probability. Following Keccak's specification[3], most parameter choices for (r, c) satisfy $c \leq r$. Given that the tight security bound for both Sponge-DRBG and POSDRBG is $O(2^{c/2}, 2^{\lambda/2})$, setting $\lambda = c$ is reasonable. Therefore, the required condition $\lambda \leq r$ for this attack is well justified. The condition ensures that the entire I fits within the r-bit region, satisfying the λ-legitimacy condition with a single refresh query.

New Security Reduction. To address this issue, we propose an alternative approach to decomposing the seedless robustness game into two subgames, dubbed Multi-Recovering (M-Rec) and Multi-Preserving (M-Pres), respectively. The reduction based on M-Rec and M-Pres ensures that the next algorithm always follows the refresh algorithm in the real world. As illustrated in Figure 1,

[3] `https://keccak.team/keccak_specs_summary.html`

the next algorithm in a permutation-based DRBG employs feed-forward, making permutation inversion infeasible. Consequently, in both games, any attack using inverse queries to the permutation is inherently blocked at the next query. As a result, our reduction remains unaffected by issues arising from an invertible refresh.

Additionally, our reduction can be seen as an extension of the recovering and the preserving games introduced in [10]. Unlike the previous approach, we use M-Rec and M-Pres to handle multiple recovering and preserving subgames, respectively, achieving tight security bounds for permutation-based DRBGs including Sponge-DRBG and POSDRBG.

2 Preliminaries

2.1 Notations and Basic Notions

NOTATION. Throughout this paper, we fix a positive integer n as the size of the underlying primitive (so a *block* is of n bits). We write 0^n to denote the n-bit string of all zeros. Given a non-empty finite set X, $x \leftarrow_{\$} X$ denotes that x is chosen uniformly at random from X. For a set X, $|X|$ denotes the number of elements in X. For two sets X and Y, $X \sqcup Y$ denotes their disjoint union. For a set $S \subseteq \{0,1\}^n$ and $x \in \{0,1\}^n$, we write $S \oplus x$ to denote $\{s \oplus x : s \in S\}$.

For a (binary) string x, $|x|$ denotes the length of x. The empty string is denoted ε, where $|\varepsilon| = 0$. For an ℓ-bit string x, and integers m and n such that $0 \le m \le n < \ell$, $x[m:n]$ denotes an $(n-m+1)$-bit string from the m-th bit to the n-th bit of x and $x[m:\cdot]$ denotes an $(\ell-m+1)$-bit string from the m-th bit to the last bit of x. When $M = M_1 \| \cdots \| M_w$ where $|M_i| = t$ for $i = 1, \ldots, w-1$, and $0 < |M_w| \le t$, we write $(M_1, \ldots, M_w) \xleftarrow{t} M$.

Given an internal state x for a permutation-based DRBG of rate r and capacity c, we use notations $x^{(r)}$ and $x^{(c)}$ to denote the leftmost r bits of x and the rightmost c bits of x, respectively. So we have $x^{(r)} = x[0 : r-1]$ and $x^{(c)} = x[|x| - c + 1 : \cdot]$.

We use & to denote the bitwise AND of two binary strings. For integers i and s such that $s \ge 2$ and $0 \le i < 2^s$, $\langle i \rangle_s$ denotes the s-bit representation of i. For a real number t, $\lceil t \rceil$ is the smallest integer that is equal to or larger than t.

IDEAL PERMUTATION MODEL. The set of all permutations of $\{0,1\}^n$ is denoted $\mathsf{Perm}(n)$. In the ideal permutation model, a permutation π is chosen uniformly at random from $\mathsf{Perm}(n)$, and an adversary is allowed oracle access to π and π^{-1}.

DISTINGUISHING GAME. We will prove the robustness security of DRBGs using a series of distinguishing games. Given two worlds $\mathcal{Z}_0$ and $\mathcal{Z}_1$, the goal of an adversary $\mathcal{A}$ is to win the following game.

1. A single bit $b \in \{0,1\}$ is chosen uniformly at random, which is unknown to $\mathcal{A}$.

2. $\mathcal{A}$ is allowed to make oracle queries to $\mathcal{Z}_b$ subject to certain rules.
3. After the interaction, $\mathcal{A}$ outputs b'. If $b' = b$, $\mathcal{A}$ wins.

The distinguishing advantage of $\mathcal{A}$ against this game is defined as follows.

$$\Delta_{\mathcal{A}}(\mathcal{Z}_0, \mathcal{Z}_1) = \left|\Pr[1 \leftarrow \mathcal{A}|b = 0] - \Pr[1 \leftarrow \mathcal{A}|b = 1]\right|.$$

H-COEFFICIENT TECHNIQUE. At the end of the distinguishing game, an adversary obtains a certain transcript, containing all the information obtained during the attack. A transcript is called *attainable* if the probability of obtaining it in the ideal world is non-zero.

Lemma 1 (Patarin's H-coefficient technique). *Let $\mathcal{T}$ be the set of all attainable transcripts. Suppose that $\mathcal{T}$ is partitioned as $\mathcal{T} = \mathcal{T}_{\mathsf{good}} \sqcup \mathcal{T}_{\mathsf{bad}}$ for two subsets $\mathcal{T}_{\mathsf{good}}$ and $\mathcal{T}_{\mathsf{bad}}$, where*

$$\mathsf{p}_{\mathsf{id}}\left[\tau \in \mathcal{T}_{\mathsf{bad}}\right] \leq \epsilon_1$$

for some $\epsilon_1 \geq 0$, and there exists $\epsilon_2 \geq 0$ such that

$$\frac{\mathsf{p}_{\mathsf{re}}\left[\tau\right]}{\mathsf{p}_{\mathsf{id}}\left[\tau\right]} \geq 1 - \epsilon_2$$

for any $\tau \in \mathcal{T}_{\mathsf{good}}$. Then for any distinguisher $\mathcal{A}$, one has

$$\Delta_{\mathcal{A}}(\mathcal{Z}_0, \mathcal{Z}_1) \leq \epsilon_1 + \epsilon_2.$$

MIN-ENTROPY. The prediction probability of a random variable X is defined as

$$\mathsf{Pred}(X) \stackrel{\text{def}}{=} \max_x \Pr\left[X = x\right].$$

Then for a random variable Y, we define

$$\mathsf{Pred}(X|y) \stackrel{\text{def}}{=} \max_x \Pr\left[X = x|Y = y\right].$$

Then conditional prediction probability of X over Y is defined as

$$\mathsf{Pred}(X|Y) \stackrel{\text{def}}{=} E_{y \leftarrow Y}(\mathsf{Pred}(X|y)),$$

and the (average-case) conditional min-entropy is defined as

$$H_\infty(X|Y) \stackrel{\text{def}}{=} -\log(\mathsf{Pred}(X|Y)).$$

2.2 Seedless Robustness Model

The seedless robustness model [8] is an adaptation of the "seeded" robustness model [10], intended to address the unrealistic assumption of the original framework, which required the presence of a random seed that must remain secret from the adversary and independent of the entropy source.

Seedless Robustness Oracle. Consider

$$\mathcal{G} = (\mathsf{refresh}[P], \mathsf{next}[P]),$$

a DRBG constructed using an ideal primitive P (e.g., a random oracle, ideal cipher, or random permutation), where P is uniformly selected from the set of all possible primitives, denoted as $\mathcal{P}$. The seedless robustness oracles are then defined as outlined in Algorithm 1, where C represents the entropy accumulated in the DRBG, and λ is a fixed parameter capturing the minimum required accumulated entropy.

Algorithm 1 Oracles for Seedless Robustness Game

Procedure INIT()
1: $b \leftarrow_{\$} \{0,1\}$
2: $P \leftarrow_{\$} \mathcal{P}$
3: $C \leftarrow 0$
4: $S \leftarrow 0^n$
5: **return** S

Procedure REF(I, γ)
1: $S \leftarrow \mathsf{refresh}[P](S, I)$
2: $C \leftarrow C + \gamma$
3: **return** γ

Procedure ROR(len)
1: $y_0 \leftarrow_{\$} \{0,1\}^{\mathsf{len}}$
2: $(S, y_1) \leftarrow \mathsf{next}[P](S, \mathsf{len})$
3: **if** $C < \lambda$ **then**
4: $\quad C \leftarrow 0$
5: $\quad$ **return** y_1
6: **else**
7: $\quad$ **return** y_b

Procedure GET();
1: $C \leftarrow 0$
2: **return** S

Procedure SET(S^*)
1: $S \leftarrow S^*$
2: $C \leftarrow 0$

Procedure $P(x)$
1: **return** $P(x)$

Procedure $P^{-1}(x)$ *//If available*
1: **return** $P^{-1}(x)$

SEEDLESS ADVERSARY. In the "seeded" model, the distribution sampler $\mathcal{D}$ supplies entropy inputs to the adversary $\mathcal{A}$, which distinguishes between two worlds in the seeded robustness game. Since $\mathcal{D}$ lacks access to the DRBG's primitive, the entropy inputs remain independent of the primitive. However, [8] pointed out this assumption is unrealistic. They renamed $\mathcal{D}$ as $\mathcal{A}_1$. The only difference between $\mathcal{D}$ in the seeded model and $\mathcal{A}_1$ in the seedless model is that $\mathcal{A}_1$ gets access to the primitive oracle. Since $\mathcal{A}_1$ can simulate using primitive queries, it gains complete knowledge of the DRBG's internal state under this definition, rendering any attack by $\mathcal{A}_1$ trivial. To address this, [8] introduces $\mathcal{A}_2$, analogous to the adversary $\mathcal{A}$ in the "seeded" model. $\mathcal{A}_2$ has no access to $\mathcal{A}_1$'s entropy input information and is responsible for distinguishing between the two worlds in the seedless robustness game. Consequently, a seedless adversary $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$ operates as follows:

- $\mathcal{A}_1$ is restricted to querying only the entropy accumulation oracle REF and the primitive oracle P (and P^{-1} if available), whereas $\mathcal{A}_2$ is permitted to access all other oracles except REF,
- $\mathcal{A}_1$ has full knowledge of the queries made by $\mathcal{A}_2$ along with their corresponding responses, whereas $\mathcal{A}_2$ can see only the responses to $\mathcal{A}_1$'s queries without knowing the queries themselves,
- The adversary $\mathcal{A}$ keeps and updates two states, S_{all} and $S_{\mathcal{A}_1}$. The state S_{all} is accessible by both $\mathcal{A}_1$ and $\mathcal{A}_2$, whereas $S_{\mathcal{A}_1}$ is accessible only by $\mathcal{A}_1$. $\mathcal{A}_1$ and $\mathcal{A}_2$ communicate through S_{all}. By separating S_{all} and $S_{\mathcal{A}_1}$, $\mathcal{A}_1$ is able to withhold its information about entropy inputs from $\mathcal{A}_2$.

Then, the seedless robustness game is defined as follows against the adversary $\mathcal{A}$.

Seedless Robustness Game

1. INIT is executed.
2. $\mathcal{A}_1$ makes queries to REF, P and P^{-1} (if available), while $\mathcal{A}_2$ makes queries to ROR, GET, SET, P and P^{-1} (if available) in an interleaved manner.
3. $\mathcal{A}_2$ outputs $b' \in \{0,1\}$.
4. If $b' = b$, then $\mathcal{A}$ wins, and $\mathcal{A}$ loses otherwise.

The game begins with the procedure INIT, which chooses a random bit $b \in \{0,1\}$ and an ideal primitive P, and initializes the internal DRBG state S as 0^n and returns it. When $\mathcal{A}_1$ makes a query to REF, the DRBG state S is updated by querying refresh with input I of entropy at least γ. When $\mathcal{A}_2$ makes a query to ROR, the state S is updated by querying next, and then y_0 and y_1 are prepared: If $C < \lambda$, then it returns y_1 regardless of b, while if $C \geq \lambda$, then y_b is returned. $\mathcal{A}_2$ uses GET to obtain the current state of the DRBG, and SET(S^*) to set the current state to S^*, where C is set to 0 since the current state is known to the adversary.

Now we can distinguish two worlds $\mathcal{Z}_0$ and $\mathcal{Z}_1$ from the seedless robustness game according to bit $b \in \{0,1\}$, and for any DRBG $\mathcal{G}$, the seedless robustness

security of $\mathcal{G}$ against $\mathcal{A}$ is defined as follows.

$$\mathbf{Adv}^{\mathcal{G}}_{rob}(\mathcal{A}) \stackrel{\text{def}}{=} \Delta_{\mathcal{A}}(\mathcal{Z}_0, \mathcal{Z}_1).$$

Let $\mathbf{Adv}^{\mathcal{G}}_{rob}(p, q_1, q_2, \sigma_1, \sigma_2, \lambda)$ denote the maximum of $\mathbf{Adv}^{\mathcal{G}}_{rob}(\mathcal{A})$ over all adversaries $\mathcal{A}$ that make at most p primitive queries, at most q_1 GET and SET queries, with the total number of entropy input blocks used in REF being at most σ_1, at most q_2 ROR queries with the total number of output blocks being at most σ_2. The parameter λ represents the min-entropy threshold, and its details will be discussed shortly.

Entropy Drain. When some information on the state is leaked to the adversary, the DRBG loses its entropy. This event is called an *entropy drain*, denoted ED. In particular, an entropy drain occurs when

- INIT is executed,
- GET or SET is queried,
- ROR is queried with $C < \lambda$.

We define the *Most Recent Entropy Drain* event as MRED.

Legitimate adversary. We first assume that the entropy strings produced by real-world entropy sources contain at least some amount of entropy, even when they depend on a primitive oracle. This assumption is expressed as the legitimate adversary in the seedless robustness model [8]. For every i-th ROR query, we define the following random variables based on the state before the query:

- $\mathcal{I}_i$: a random variable for all entropy inputs, $I_i := (I_{i,1}, I_{i,2}, \ldots, I_{i,\ell_i})$, gathered from the MRED up to the i-th ROR query where ℓ_i is the number of REF queries between them,
- $\mathcal{T}_i$: a random variable comprising all query-response pairs except I_i,
- $\mathcal{S}_{all,i}$: a random variable for the state S_{all} exactly before the i-th ROR query.

$\mathcal{A}$ is called γ^*-*legitimate* if

$$H_\infty(\mathcal{I}_i|\mathcal{T}_i, \mathcal{S}_{all,i}) \geq \sum_{j=1}^{\ell_i} \gamma_{i,j} \geq \gamma^*$$

holds for every i, where $\gamma_{i,j}$ is the return value of the query $\mathrm{REF}(I_{i,j}, \gamma_{i,j})$. Note that $S_{\mathcal{A}_1}$ is excluded from the definition of $\mathcal{S}_{all,i}$. In Algorithm 1, the entropy within the DRBG is tracked by incrementing the counter C with the entropy γ of each input I. According to the λ-legitimacy condition, the adversary $\mathcal{A}_1$ must provide entropy inputs with min-entropy greater than C from the perspective of $\mathcal{A}_2$ at that point. Consequently, if $C \geq \lambda$, the total accumulated entropy provided by $\mathcal{A}_1$ must also be at least λ. In summary, $\mathcal{A}_1$ is required to generate entropy inputs that satisfy the λ-legitimacy condition, ensuring that $\mathcal{A}_2$ can guess the entropy inputs with a probability of no more than $2^{-\lambda}$.

Canonical adversary. An λ-*legitimate* adversary $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$ is referred to as *canonical* if it adheres to the following rules.

Algorithm 2 Oracles for $\mathcal{Z}_h$

Procedure ROR*(len)
if $C < \lambda$ **then**
  $(S, y) \leftarrow \mathsf{next}[P](S, \mathsf{len})$
  $C \leftarrow 0$
else
  $S \leftarrow_\$ \{0,1\}^n$
  $y \leftarrow_\$ \{0,1\}^{\mathsf{len}}$
return y

1. $\mathcal{A}_2$ makes queries to ROR only when $C \geq \lambda$.
2. Between the MRED and the first ROR query thereafter, $\mathcal{A}_2$ does not make a query to GET or SET if $C > 0$.

We can assume that the seedless robustness adversary $\mathcal{A}$ is canonical since two worlds $\mathcal{Z}_0$ and $\mathcal{Z}_1$ are distinguished only when a query is made to ROR with $C \geq \lambda$; for any adversary $\mathcal{A}$ that violates the above rules, a canonical adversary $\mathcal{A}'$ can be constructed using $\mathcal{A}$ as a subroutine without degrading $\mathcal{A}$'s distinguishing advantage since $\mathcal{A}'$ can faithfully simulate the responses to the queries that $\mathcal{A}$ makes without following the above rules by making at most $\sigma_1 + \sigma_2$ additional queries to the underlying primitives, where σ_1 is a maximum total number of entropy input blocks in every REF, and σ_2 maximum total number of output blocks in every ROR. We refer to [6,8] for more details. From now on, we assume every adversary is **canonical** and **λ-legitimate**.

3 Our Reduction Technique

In this section, we introduce our new reduction technique based on the Multi-Recovering (M-Rec) and Multi-Preserving (M-Pres) security games. Specifically, our proof consists of the following two steps.

1. We define a hybrid world $\mathcal{Z}_h$ to realize the ideal state updates for each game, and by Lemma 2, we establish
$$\Delta_{\mathcal{A}}(\mathcal{Z}_h, \mathcal{Z}_0) \leq \Delta_{\mathcal{A}'}(\mathcal{Z}_h, \mathcal{Z}_1),$$
where $\mathcal{Z}_0$ (resp. $\mathcal{Z}_1$) is the ideal (resp. real) world of the seedless robustness game.
2. In order to upper bound $\Delta_{\mathcal{A}'}(\mathcal{Z}_h, \mathcal{Z}_1)$, we introduce the M-Rec and M-Pres security games, and show that Multi-Recovering and Multi-Preserving security implies seedless robustness security by Lemma 4.

Defining a Hybrid World. Since M-Pres requires the initial state to be random, we define a hybrid world $\mathcal{Z}_h$. This hybrid world, $\mathcal{Z}_h$, is derived from the real world $\mathcal{Z}_1$ by replacing ROR with ROR*, as detailed in Algorithm 2. When

sufficient entropy is accumulated in the state, ROR* randomly generates both state and output bits. In contrast, in the ideal world of the Algorithm 1, ROR produces only random output bits, while the state is updated using the next function. Then, by the triangle inequality, we have

$$\mathbf{Adv}^{\mathcal{G}}_{rob}(\mathcal{A}) = \Delta_{\mathcal{A}}(\mathcal{Z}_0, \mathcal{Z}_1) \leq \Delta_{\mathcal{A}}(\mathcal{Z}_0, \mathcal{Z}_h) + \Delta_{\mathcal{A}}(\mathcal{Z}_h, \mathcal{Z}_1). \tag{1}$$

For any adversary $\mathcal{A}$, its oracle's random coin $b \in \{0,1\}$ for $\Delta_{\mathcal{A}}(\mathcal{W}_0, \mathcal{W}_1)$ captures the following: if $b = 0$ (resp. $b = 1$), then $\mathcal{A}$ is interacting with $\mathcal{W}_0$ (resp. $\mathcal{W}_1$) where $\mathcal{W}_0, \mathcal{W}_1 \in \{\mathcal{Z}_0, \mathcal{Z}_1, \mathcal{Z}_h\}$. The decision bit b' of $\mathcal{A}$ is defined similarly. Note that $\Delta_{\mathcal{A}}(\mathcal{W}_0, \mathcal{W}_1) = \Delta_{\mathcal{A}}(\mathcal{W}_1, \mathcal{W}_0)$ by the definition. Then we have the following lemma:

Lemma 2. *Given any distinguisher $\mathcal{A}$ distinguishing $\mathcal{Z}_h$ and $\mathcal{Z}_0$, one can construct a distinguisher $\mathcal{A}'$ distinguishing $\mathcal{Z}_h$ and $\mathcal{Z}_1$ using $\mathcal{A}$ as a subroutine, where $\mathcal{A}'$ makes queries as many as $\mathcal{A}$, and*

$$\Delta_{\mathcal{A}}(\mathcal{Z}_h, \mathcal{Z}_0) \leq \Delta_{\mathcal{A}'}(\mathcal{Z}_h, \mathcal{Z}_1).$$

Proof. $\mathcal{A}'$ outputs the decision bit b' from $\mathcal{A}$. To have the same advantage as $\mathcal{A}$, $\mathcal{A}'$ must simulate the two worlds, $\mathcal{Z}_h$ and $\mathcal{Z}_0$, for $\mathcal{A}$ while it interacts with $\mathcal{Z}_h$ and $\mathcal{Z}_1$. The simulation strategy employed by $\mathcal{A}'$ is as follows:

$\mathcal{A}'$ forwards $\mathcal{A}$'s oracle queries to its oracles and relays the responses back to $\mathcal{A}$, except when $\mathcal{A}$ queries ROR with $C \geq \lambda$. In that case, $\mathcal{Z}_0$ and $\mathcal{Z}_h$ generate random output bits, while $\mathcal{Z}_1$ produces the DRBG's output. To ensure simulation, $\mathcal{A}'$ randomly samples a bitstring and returns it to $\mathcal{A}$ whenever $\mathcal{A}$ queries ROR with $C \geq \lambda$. Finally, $\mathcal{A}'$ outputs the result from its subroutine $\mathcal{A}$. By this method, $\mathcal{A}'$ simulates $\mathcal{Z}_h$ (resp. $\mathcal{Z}_0$) for $\mathcal{A}$ when the random coin $b = 0$ (resp. $b = 1$), thereby completing the proof.

Upper bounding $\Delta_{\mathcal{A}}(\mathcal{Z}_1, \mathcal{Z}_h)$. In order to upper bound $\Delta_{\mathcal{A}}(\mathcal{Z}_1, \mathcal{Z}_h)$, we define an intermediate world R (see Figure 2).

- R is exactly the same as $\mathcal{Z}_1$, except that whenever C reaches λ (so $C \geq \lambda$), the first ROR query made thereafter is answered with ROR*(len).

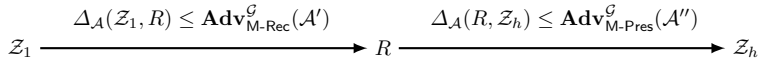


Fig. 2: Game hopping between the worlds

As the next step, we upper bound the adversarial distinguishing advantage for each pair of consecutive worlds in Figure 2. We will define M-Rec and M-Pres games, where the hardness of each distinguishing game is reduced to the hardness of one of the two games. These subgames capture the following security properties:

- Multi-recovering (M-Rec) security ensures that the state and the output of the DRBG remain indistinguishable from uniform ones once a sufficient amount of entropy is absorbed following the entropy drain event. Unlike Recovering security, which addresses only a single case between the MRED and the first query to ROR after the MRED [8], M-Rec extends this guarantee to all such cases.
  - In detail, each Recovering game is designed to deal with a subsequence of queries beginning with an entropy drain, followed by a series of REF queries, and ending with an ROR query. So, each Recovering game can be modeled in a way that the M-Rec game adversary is allowed to query M-REC only once. Note that within the entire robustness game, the adversary may perform this type of query subsequence multiple times. Therefore, multiple rounds of game hopping are required to use a hybrid argument based on a single Recovering game. In contrast, the M-Rec game allows the adversary to perform multiple M-REC queries, hence it does not require multiple rounds of game hopping.
- Multi-preserving (M-Pres) security ensures that the state and the output of the DRBG remain indistinguishable from uniform ones when the initial state is random, even after absorbing arbitrary entropy inputs. Similarly to M-Rec, M-Pres merges all Preserving cases described in [8] into a single game.

For each game, we define the corresponding adversary $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$ similarly to the seedless robustness adversary. $\mathcal{A}_1$ generates the entropy inputs $I_1, \ldots, I_\ell$, and $\mathcal{A}_2$ attempts to guess the challenge bit b'. For the M-Rec game, we impose a λ-legitimacy condition on each M-REC query, requiring that the entropy inputs $I_1, \ldots, I_\ell$ contain at least λ bits of min-entropy. For the M-Pres game where the DRBG is assumed to have a sufficient amount of accumulated entropy, the λ-legitimacy condition is not required for $\mathcal{A}$.

First, the M-Rec game is defined as follows, using the oracles defined in Algorithm 3. Note that the M-Rec game aims to capture the property that a DRBG regains its randomness after the entropy drain event when it absorbs sufficient entropy. Therefore, we allow the adversary $\mathcal{A}_2$ to specify an initial state S_0 for M-REC to model the entropy drain. $\mathcal{A}_1$ is solely responsible for generating entropy inputs, subject to the λ-legitimacy condition on each invocation of M-REC. Hence, we do not allow $\mathcal{A}_2$ to access entropy inputs.

M-Rec **Game**

1. INIT is executed.
2. $\mathcal{A}$ makes queries to M-REC, and P (and P^{-1}). A query to M-REC is made as follows.
   (a) $\mathcal{A}_2$ chooses an initial state S_0 and an output length len, and then return them to $\mathcal{A}_1$ through S_{all}.
   (b) $\mathcal{A}_1$ chooses entropy inputs $I_1, \ldots, I_\ell$, each of which is of r bits, makes queries M-REC$(S_0, I_1 \parallel \cdots \parallel I_\ell, \mathsf{len})$ and then passes the responses to $\mathcal{A}_2$ through S_{all}.

3. $\mathcal{A}_2$ outputs $b' \in \{0,1\}$.
4. If $b' = b$, then $\mathcal{A}$ wins, and $\mathcal{A}$ loses otherwise.

Algorithm 3 Oracles for M-Rec game

Procedure INIT()
1: $b \leftarrow_{\$} \{0,1\}$
2: $P \leftarrow_{\$} \mathcal{P}$

Procedure M-REC(S, I, len)
1: **if** $b = 0$ **then**
2: $S \parallel y \leftarrow_{\$} \{0,1\}^{n+\mathsf{len}}$
3: **else**
4: $(I_1, \ldots, I_\ell) \xleftarrow{r} I$
5: **for** $i \leftarrow 1$ to ℓ **do**
6: $S \leftarrow \mathsf{refresh}[P](S, I_i)$
7: $(S, y) \leftarrow \mathsf{next}[P](S, \mathsf{len})$
8: **return** (S, y)

Procedure $P(x)$
1: **return** $P(x)$

Procedure $P^{-1}(y)$ *//If available*
1: **return** $P^{-1}(y)$

For a DRBG $\mathcal{G}$, let $\mathbf{Adv}^{\mathcal{G}}_{\mathsf{M\text{-}Rec}}(p, q, \sigma_1, \sigma_2, \lambda)$ denote the maximum winning probability over all the canonical λ-legitimate adversaries $\mathcal{A}$ that make at most p queries to P (and P^{-1}) and at most q M-REC queries, where the total length of the entropy input I is at most σ_1 blocks, and total length of the M-REC output y is at most σ_2 blocks.

Next, we define the M-Pres game as follows, using the oracles defined in Algorithm 4. Note that the M-Pres game is designed to capture the property that a DRBG preserves its randomness as long as its state contains sufficient entropy, even if the adversary maliciously crafts the entropy input. Therefore, a legitimacy condition is not required for this game. Each interval between two INTER queries constitutes a single Preserving case. Since M-Pres encompasses multiple Preserving cases, it should simulate both the termination of one Preserving case and the initiation of the next. To achieve this, we define the INTER operation to mark the end of the previous Preserving case and the start of the subsequent one. A Preserving case ends when an entropy drain is triggered. When the entropy drain occurs as a result of GET(), the response to GET() should be made by accessing the state at the time of the drain. Therefore, S_p is should be returned. Additionally, INTER resets the state S to a uniformly random value, since each Preserving case begins with a uniformly random state.

M-Pres **Game**

1. INIT is executed.

2. $\mathcal{A}_1$ makes queries to $\mathrm{REF}_{\mathrm{mpres}}$ and P (and P^{-1}), while $\mathcal{A}_2$ makes queries to INTER, $\mathrm{ROR}_{\mathrm{mpres}}$ and P (and P^{-1}).
3. $\mathcal{A}_2$ outputs $b' \in \{0,1\}$.
4. If $b' = b$, then $\mathcal{A}$ wins, and $\mathcal{A}$ loses otherwise.

Algorithm 4 Oracles for M-Pres game

Procedure INIT()
1: $b \leftarrow_{\$} \{0,1\}$
2: $P \leftarrow_{\$} \mathcal{P}$
3: $S \leftarrow_{\$} \{0,1\}^n$

Procedure $\mathrm{REF}_{\mathrm{mpres}}(I)$
1: $S \leftarrow \mathsf{refresh}[P](S, I)$

Procedure $P(x)$
1: **return** $P(x)$

Procedure INTER()
1: $S_p \leftarrow S$
2: $S \leftarrow_{\$} \{0,1\}^n$
3: **return** S_p

Procedure $\mathrm{ROR}_{\mathrm{mpres}}(\mathsf{len})$
1: **if** $b = 0$ **then**
2: $\quad S \parallel y \leftarrow_{\$} \{0,1\}^{n+\mathsf{len}}$
3: **else**
4: $\quad (S, y) \leftarrow \mathsf{next}[P](S, \mathsf{len})$
5: **return** y

Procedure $P^{-1}(y)$ *//If available*
1: **return** $P^{-1}(y)$

For a DRBG $\mathcal{G}$, let $\mathbf{Adv}^{\mathcal{G}}_{\mathsf{M\text{-}Pres}}(p, q, \sigma_1, \sigma_2)$ denote the maximum winning probability over all the canonical adversaries $\mathcal{A}$ that make at most p queries to P (and P^{-1}) and at most q queries to INTER and at most σ_1 queries to $\mathrm{REF}_{\mathrm{mpres}}$, where the total length of the outputs from $\mathrm{ROR}_{\mathrm{mpres}}$ is at most σ_2 blocks.

The distinguishing advantage for the worlds in Figure 2 is upper bounded by the winning probabilities of the M-Rec and M-Pres games as follows.

Lemma 3. *Suppose a distinguisher $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$ makes at most p primitive queries, at most q_1* GET *and* SET *queries, with the total number of entropy input blocks used in* REF *being at most σ_1, at most q_2* ROR *queries with the total number of output blocks being at most σ_2. Then one has*

$$\Delta_{\mathcal{A}}(\mathcal{Z}_1, R) \leq \mathbf{Adv}^{\mathcal{G}}_{\mathsf{M\text{-}Rec}}(p + \sigma_1 + \sigma_2, q_2, \sigma_1, \sigma_2, \lambda),$$
$$\Delta_{\mathcal{A}}(R, \mathcal{Z}_h) \leq \mathbf{Adv}^{\mathcal{G}}_{\mathsf{M\text{-}Pres}}(p + \sigma_1, q_1, \sigma_1, \sigma_2).$$

Proof. Given $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$ whose goal is to distinguish $\mathcal{Z}_1$ and R, we construct an adversary $\mathcal{A}' = (\mathcal{A}'_1, \mathcal{A}'_2)$ against the M-Rec game, where $\mathcal{A}'_1$ and $\mathcal{A}'_2$ use $\mathcal{A}_1$ and $\mathcal{A}_2$ as subroutines, respectively. $\mathcal{A}'$ outputs the decision bit b' from $\mathcal{A}$. To have the same advantage as $\mathcal{A}$, $\mathcal{A}'$ must simulate the two worlds, $\mathcal{Z}_h$ and R, for $\mathcal{A}$ while it interacts with M-Rec oracles. In the simulation, $\mathcal{A}'_1$ has access to $\mathcal{S}_{all'}$

and $\mathcal{S}_{\mathcal{A}_1'}$, while $\mathcal{A}_2'$ has access to $\mathcal{S}_{all'}$, following the definition of the seedless adversary. Now, we show that $\mathcal{A}'$ simulates $\mathcal{Z}_1$ and R for $\mathcal{A}$ by responding to all of its queries as follows:

- A states $\mathcal{S}_{all'}$ and $\mathcal{S}_{\mathcal{A}_1'}$ of the adversary $\mathcal{A}'$ are defined as follows:

$$\mathcal{S}_{all'} := \{S_{\text{M-REC}}, y', C, \gamma', x\}, \qquad \mathcal{S}_{\mathcal{A}_1'} := \{I_s\}.$$

  Variable x is used to check if an ROR query is the first made after the condition $C \geq \lambda$ is satisfied. If $x = 1$, then it is the first such query.
- Firstly, $\mathcal{A}_1'$ sets $S_{\text{M-REC}} \leftarrow 0^n$, $y' \leftarrow \epsilon$, $C \leftarrow 0$, $\gamma' \leftarrow 0$, $I_s \leftarrow \epsilon$, and $x \leftarrow 0$.
- Primitive queries: $\mathcal{A}_1'$ (resp. $\mathcal{A}_2'$) relays the queries made by $\mathcal{A}_1$ (resp. $\mathcal{A}_2$) to the primitive oracle given by the M-Rec game, and faithfully relays the responses to $\mathcal{A}_1$ (resp. $\mathcal{A}_2$).
- REF(I, γ):
  1. $\mathcal{A}_1'$ sets $C \leftarrow C + \gamma$, $\gamma' \leftarrow \gamma$, and $I_s \leftarrow I_s \parallel I$.
  2. $\mathcal{A}_2'$ returns γ' to $\mathcal{A}_2$.
- ROR(len) with $x = 0$:
  1. $\mathcal{A}_1'$ makes a query M-REC$(S_{\text{M-REC}}, I_s, \mathsf{len})$, and obtains the response (S, y).
  2. $\mathcal{A}_1'$ sets $S_{\text{M-REC}} \leftarrow S$, $y' \leftarrow y$, $I_s \leftarrow \epsilon$, and $x \leftarrow 1$.
  3. $\mathcal{A}_2'$ returns y' to $\mathcal{A}_2$.
- ROR(len) with $x = 1$:
  1. $\mathcal{A}_1'$ makes primitive queries to simulate REF queries with $S_{\text{M-REC}}$ and I_s, then updates $S_{\text{M-REC}}$ and sets $I_s \leftarrow \epsilon$.
  2. $\mathcal{A}_2'$ makes primitive queries to simulate ROR(len) with $S_{\text{M-REC}}$, obtains (S, y). $\mathcal{A}_2'$ sets $S_{\text{M-REC}} \leftarrow S$ and $y' \leftarrow y$, and returns y' to $\mathcal{A}_2$.
- GET():
  1. $\mathcal{A}_1'$ makes primitive queries to simulate REF queries with $S_{\text{M-REC}}$ and I_s, and updates $S_{\text{M-REC}}$.
  2. $\mathcal{A}_2'$ returns $S_{\text{M-REC}}$ to $\mathcal{A}_2$.
  3. $\mathcal{A}_1'$ sets $C \leftarrow 0$, $I_s \leftarrow \varepsilon$, and $x \leftarrow 0$.
- SET(S): $\mathcal{A}_1'$ sets $S_{\text{M-REC}} \leftarrow S$, $C \leftarrow 0$, $I_s \leftarrow \varepsilon$, and $x \leftarrow 0$.
- At the end of the game, $\mathcal{A}_2'$ returns the same output as $\mathcal{A}_2$.

Then $\mathcal{A}'$ becomes a λ-legitimate adversary against the M-Rec game that makes at most $p + \sigma_1 + \sigma_2$ primitive queries, at most q_2 queries to M-REC, where the total length of the entropy input I is at most σ_1 blocks, and the total length of the M-REC output y is at most σ_2 blocks. Furthermore, $\mathcal{A}'$ has the same advantage as $\mathcal{A}$. Therefore we have

$$\Delta_{\mathcal{A}}(\mathcal{Z}_1, R) \leq \mathbf{Adv}^{\mathcal{G}}_{\mathsf{M\text{-}Rec}}(p + \sigma_1 + \sigma_2, q_2, \sigma_1, \sigma_2, \lambda).$$

Next, suppose that $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$ tries to distinguish R and $\mathcal{Z}_h$. We can construct an adversary $\mathcal{A}'' = (\mathcal{A}_1'', \mathcal{A}_2'')$ againt the M-Pres game, where $\mathcal{A}_1''$ and

$\mathcal{A}_2''$ use $\mathcal{A}_1$ and $\mathcal{A}_2$ as subroutines, respectively. In the simulation, $\mathcal{A}_1''$ has access to $\mathcal{S}_{all''}$ and $\mathcal{S}_{\mathcal{A}_1''}$, while $\mathcal{A}_2''$ has access to $\mathcal{S}_{all''}$, following the definition of the seedless adversary. Now, we show that $\mathcal{A}''$ simulates R and $\mathcal{Z}_h$ for $\mathcal{A}$ using the M-Pres oracles by answering all of its queries as follows:

- A states $\mathcal{S}_{all''}$ and $\mathcal{S}_{\mathcal{A}_1''}$ of the adversary $\mathcal{A}''$ are defined as follows:

$$\mathcal{S}_{all''} := \{S_{\text{M-Pres}}, C, \gamma', \overline{x}\}, \qquad \mathcal{S}_{\mathcal{A}_1''} := \{I_s\}.$$

  Variable $\overline{x}$ is used to check if an ROR query is **not** the first made after the condition $C \geq \lambda$ is satisfied. If $\overline{x} = 1$, then it is not the first such query.
- Firstly, $\mathcal{A}_1''$ sets $S_{\text{M-Pres}} \leftarrow 0^n$, $C \leftarrow 0$, $\gamma' \leftarrow 0$, $I_s \leftarrow \epsilon$, and $\overline{x} \leftarrow 0$.
- Primitive queries: $\mathcal{A}_1''$ (resp. $\mathcal{A}_2''$) relays the queries made by $\mathcal{A}_1$ (resp. $\mathcal{A}_2$) to the primitive oracle given by the M-Pres game, and faithfully relays the responses to $\mathcal{A}_1$ (resp. $\mathcal{A}_2$).
- REF(I, γ):
  1. $\mathcal{A}_1''$ sets $C \leftarrow C + \gamma$, $\gamma' \leftarrow \gamma$, and $I_s \leftarrow I_s \parallel I$.
  2. $\mathcal{A}_2''$ returns γ' to $\mathcal{A}_2$.
- ROR(len) with $\overline{x} = 0$:
  1. $\mathcal{A}_2''$ generates (S, y) of $(n + \mathsf{len})$ bits uniformly at random, sets $S_{\text{M-Pres}} \leftarrow S$, and returns y to $\mathcal{A}_2$.
  2. $\mathcal{A}_1''$ sets $I_s \leftarrow \epsilon$, and $\overline{x} \leftarrow 1$.
- ROR(len) with $\overline{x} = 1$:
  1. $\mathcal{A}_1''$ makes queries to $\text{REF}_{\text{mpres}}$ with I_s, and sets $I_s \leftarrow \epsilon$.
  2. $\mathcal{A}_2''$ makes a query $\text{ROR}_{\text{mpres}}(\mathsf{len})$, obtains y as the response, and returns y to $\mathcal{A}_2$.
- GET() with $\overline{x} = 0$: $\mathcal{A}_2''$ returns $S_{\text{M-Pres}}$ to $\mathcal{A}_2$. (Since $\mathcal{A}$ is canonical, this query is the first made after an entropy drain event.)
- GET() with $\overline{x} = 1$:
  1. $\mathcal{A}_1''$ makes queries to $\text{REF}_{\text{mpres}}$ with I_s, and sets $I_s \leftarrow \epsilon$.
  2. $\mathcal{A}_2''$ makes a query to INTER, obtaining S_p, and returns S_p to $\mathcal{A}_2$. Then, $\mathcal{A}_2''$ sets $S_{\text{M-Pres}} \leftarrow S_p$.
  3. $\mathcal{A}_1''$ sets $C \leftarrow 0$, and $\overline{x} \leftarrow 0$.
- SET(S) with $\overline{x} = 0$: $\mathcal{A}_2''$ sets $S_{\text{M-Pres}} \leftarrow S$. (Since $\mathcal{A}$ is canonical, this query is the first made after an entropy drain event.)
- SET(S) with $\overline{x} = 1$:
  1. $\mathcal{A}_2''$ makes a query to INTER and sets $S_{\text{M-Pres}} \leftarrow S$.
  2. $\mathcal{A}_1''$ sets $C \leftarrow 0$, $I_s \leftarrow \epsilon$, and $\overline{x} \leftarrow 0$.
- At the end of the game, $\mathcal{A}_2''$ returns the same output as $\mathcal{A}_2$.

Then $\mathcal{A}''$ becomes an adversary against the M-Pres game that makes at most p primitive queries, at most q_1 queries to INTER and at most σ_1 queries to $\text{REF}_{\text{mpres}}$, where the total length of the outputs from $\text{ROR}_{\text{mpres}}$ is at most σ_2 blocks. Note that, for REF, we have two cases:

Algorithm 5 Syntax of the Sponge-DRBG

$\mathsf{refresh} : \{0,1\}^n \times \{0,1\}^r \to \{0,1\}^n$
Procedure $\mathsf{refresh}[\pi](S, I)$
1: $S \leftarrow \pi\left(S^{(r)} \oplus I \parallel S^{(c)}\right)$
2: **return** S

$\mathsf{next} : \{0,1\}^n \times \{0,1\}^* \to \{0,1\}^n \times \{0,1\}^*$
Procedure $\mathsf{next}[\pi](S, \mathsf{len})$
1: $\ell \leftarrow \lceil \mathsf{len}/r \rceil$
2: $\mathsf{out} \leftarrow \epsilon$
3: **for** $i \leftarrow 1$ to ℓ **do**
4: $\quad y \leftarrow S^{(r)}$
5: $\quad S \leftarrow \pi(S) \oplus 0^r \parallel S^{(c)}$
6: $\quad \mathsf{out} \leftarrow \mathsf{out} \parallel y$
7: **return** $(S, \mathsf{out}[0 : \mathsf{len} - 1])$

- REF is queried when $\overline{x} = 0$: since ROR with $\overline{x} = 0$ is ROR* in both worlds R and $\mathcal{Z}_h$, it will output random bits regardless of the state of the DRBG. Hence, the REF query does not need to be simulated.
- REF is queried when $\overline{x} = 1$: in the M-Pres game, $\mathcal{A}''$ can access the $\mathrm{REF}_{\mathrm{mpres}}$ oracle. Therefore, the REF query is simulated by using the $\mathrm{REF}_{\mathrm{mpres}}$ oracle without using the primitive query.

Hence, $\mathcal{A}''$ makes at most p primitive queries. Furthermore, $\mathcal{A}''$ has the same advantage as $\mathcal{A}$. Therefore we have

$$\Delta_{\mathcal{A}}(R, \mathcal{Z}_h) \leq \mathbf{Adv}^{\mathcal{G}}_{\mathsf{M\text{-}Pres}}(p, q_1, \sigma_1, \sigma_2). \qquad \square$$

Putting pieces together. By combining (1), Lemma 2, and Lemma 3, we complete the proof of the following Lemma.

Lemma 4. *Suppose that a distinguisher $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$ makes at most p primitive queries, at most q_1* GET *and* SET *queries, with the total number of entropy input blocks used in* REF *being at most σ_1, at most q_2* ROR *queries with the total number of output blocks being at most σ_2. Then one has*

$$\mathbf{Adv}^{\mathcal{G}}_{rob}(\mathcal{A}) \leq 2\mathbf{Adv}^{\mathcal{G}}_{\mathsf{M\text{-}Rec}}(p + \sigma_1 + \sigma_2, q_2, \sigma_1, \sigma_2, \lambda) + 2\mathbf{Adv}^{\mathcal{G}}_{\mathsf{M\text{-}Pres}}(p, q_1, \sigma_1, \sigma_2).$$

4 Proving Stronger Security for Sponge-DRBG

In this section, we revisit Sponge-DRBG discussed in [8], and prove its stronger security. Sponge-DRBG, proposed in [8], is a DRBG based on the Sponge construction. Sponge-DRBG is defined as in Algorithm 5 (see also Figure 1). For Sponge-DRBG, we have the following lemma 5. The proof of (2) and (3) are deferred to the following subsection 4.1 and 4.2.

Lemma 5. *If* $p + \lceil \frac{n}{r} \rceil(\sigma_1 + \sigma_2) \leq 2^{c-1}$*, then one has*

$$\begin{aligned}
&\mathbf{Adv}^{\mathsf{Sponge\text{-}DRBG}}_{\mathsf{M\text{-}Rec}}(p, q, \sigma_1, \sigma_2, \lambda) \\
&\leq \frac{2p(p+1) + 5\lceil \frac{n}{r} \rceil(\sigma_1 + \sigma_2)\left(p + \lceil \frac{n}{r} \rceil(\sigma_1 + \sigma_2)\right)}{2^c} + \frac{pq}{2^\lambda}, \qquad (2) \\
&\mathbf{Adv}^{\mathsf{Sponge\text{-}DRBG}}_{\mathsf{M\text{-}Pres}}(p, q, \sigma_1, \sigma_2) \\
&\leq \frac{\left(2q + 6\lceil \frac{n}{r} \rceil(\sigma_1 + \sigma_2)\right)\left(p + \lceil \frac{n}{r} \rceil(\sigma_1 + \sigma_2)\right) + q^2}{2^c}. \qquad (3)
\end{aligned}$$

Then, by Lemma 4 and Lemma 5, we have the following theorem.

Theorem 1. *If* $p + \lceil \frac{n}{r} \rceil(\sigma_1 + \sigma_2) \leq 2^{c-1}$*, then one has*

$$\begin{aligned}
&\mathbf{Adv}^{\mathsf{Sponge\text{-}DRBG}}_{rob}(p, q_1, q_2, \sigma_1, \sigma_2, \lambda) \\
&\leq \frac{2(p + 2\lceil \frac{n}{r} \rceil(\sigma_1 + \sigma_2))(2p + 13\lceil \frac{n}{r} \rceil(\sigma_1 + \sigma_2) + 2q_1 + 2) + 2q_1^2}{2^c} \\
&+ \frac{2(p + \lceil \frac{n}{r} \rceil(\sigma_1 + \sigma_2))q_2}{2^\lambda}.
\end{aligned}$$

4.1 Proof of (2)

Throughout the proof, we will use $\sigma := \lceil \frac{n}{r} \rceil(\sigma_1 + \sigma_2)$. In the $\mathsf{M\text{-}Rec}$ game, the adversary interacts with primitive oracles π, π^{-1} and an M-REC oracle. Let τ_p denote the transcript of queries made to the primitive oracle, and τ_q denote the transcript of queries made to the M-REC oracle. Suppose that, after the adversary finishes oracle queries, the adversary $\mathcal{A}_2$ can obtain every entropy input made by $\mathcal{A}_1$, as well as the residual values of each M-REC output $y_i[\mathsf{len}_i :]$. Note that in the ideal world, the value $y_i[\mathsf{len}_i :]$ does not exist, so it is selected uniformly at random. This only increases the adversary's advantage. Then, a transcript $\tau = (\tau_p, \tau_q)$ where

$$\begin{aligned}
\tau_p &= \{(u_i, v_i)\}_{i=1,\ldots,p}, \\
\tau_q &= \{\mathcal{Q}_1, \cdots, \mathcal{Q}_q\},
\end{aligned}$$

is defined as follows.

- τ_p: primitive query results for the permutation π, an element (u_i, v_i) satisfies

$$\pi(u_i) = v_i$$

  for $1 \leq i \leq p$.
- τ_q: for $1 \leq i \leq q$, $\mathcal{Q}_i = (S_{i,0}, I_{i,1}, \ldots, I_{i,\ell_i}, S_{i,\ell_i+w_i}, y_i)$ is a tuple of query history of i-th M-REC call where
  - $S_{i,0}$ be a initial DRBG state of i-th M-REC that adversary choose.
  - ℓ_i be the number of r-bit unit entropy inputs in i-th M-REC.
  - len_i be the required output length of i-th M-REC.

- $w_i \leftarrow \lceil \mathsf{len}_i / r \rceil$ be the r-bit unit length of i-th M-REC output y_i.
- S_{i,ℓ_i+w_i} be returned DRBG state of i-th M-REC in (4).
- $y_i = y_i[0 : \mathsf{len}_i - 1] \| y_i[\mathsf{len}_i :] \in \{0,1\}^{rw_i}$ be i-th M-REC output combined with given residual value $y_i[\mathsf{len}_i :]$ in (4).

$$(S_{i,\ell_i+w_i}, y_i[0 : \mathsf{len}_i - 1]) \leftarrow \text{M-REC}(S_{i,0}, I_{i,1} \| \cdots \| I_{i,\ell_i}, \mathsf{len}_i). \quad (4)$$

Since there is no entropy input in next, we define $I_{j,i} := 0^r$ for $\ell_j + 1 \le i \le \ell_j + w_j$ for each $\mathcal{Q}_j$. Then define ℓ_j^c as the maximum number that satisfies $(X_{j,i-1} \oplus I_{j,i} \| 0^c, X_{j,i}) \in \tau_p$ for some values $X_{j,1}, \ldots, X_{j,\ell_j^c}$ with $S_{j,0} = X_{j,0}$ for all $1 \le i \le \ell_j^c \le \ell_j$.

Bad event analysis. We define bad events for the M-Rec game as follows.

- bad_1: For $1 \le i \le p$, there exists $(u_i, v_i) \in \tau_p$ such that $u_i^{(c)} = v_i^{(c)}$.
- bad_2: For $1 \le i \le p$, there exists $(u_i, v_i) \in \tau_p$ such that $u_i^{(c)} \in V_{i-1} \wedge v_i^{(c)} \in V_{i-1}$ where

$$V_{i-1} := \left\{ u_j^{(c)} \mid (u_j, \cdot) \in \tau_p \vee (\cdot, u_j) \in \tau_p, 1 \le j < i \right\}.$$

- bad_3^j for $1 \le j \le q$: $\ell_j^c = \ell_j$.

Overview for bad *event analysis.* In the M-Rec game, a λ-legitimate adversary $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$ selects an initial state S_0 and entropy inputs $I_1, \ldots, I_\ell$ for each M-REC query. Hence, $\mathcal{A}$ is able to simulate refresh, which is used in the M-REC oracle when $b = 1$, using primitive queries. However, by the legitimacy condition, the probability of choosing I is at most $1/2^\lambda$ for any I, and it is not feasible for $\mathcal{A}$ to make primitive queries with every possible input I. Therefore, $\mathcal{A}$ is not able to simulate every refresh in any single M-REC query (resulting in bad_3) with high probability. Formally, we define a *potential chain* that can be seen as an adversarial simulation of the M-REC oracle. Note that in Sponge-DRBG, the entropy inputs are first XORed into the upper r bits of the state, which is then passed through the permutation. Two n-bit states can therefore be linked in a potential chain if the lower c bits are identical. As such, we use the lower c bits of the state to bound the maximum number of potential chains, following the approach in [8,15]. A graph G is defined using the lower c bits of the states. Then, we upper bound the probability of having such a potential chain using a full entropy input I under the legitimacy condition with the graph G (Claims 1 and 2).

The event bad_3^j implies that the adversary knows all the entropy inputs used in the j-th M-REC query, enabling a distinguishing attack. To calculate the probability of bad_3, the events bad_1 and bad_2 are required. Using a randomness of π, we can bound probabilities of the bad_1as follows:

$$\mathsf{p}_{\mathsf{id}}[\mathsf{bad}_1] \le \frac{p \cdot 2^r}{2^n - p} \le \frac{2p}{2^c},$$

where $p + \sigma \le 2^{c-1}$, we use $p \le 2^{n-1}$ for the bound.

For bad_2, consider the case where $(u_i, v_i) \in \tau_p$. Since the adversary can choose either u_i or v_i to be in V_{i-1}, the randomness is confined to only one of them. Thus, the denominator is $2^n - p$, representing the minimal possible choices for one of them. Given that there are at most $2(i-1)$ elements in V_{i-1}, the probability is calculated as follows:

$$\mathsf{p}_{\mathsf{id}}\left[\mathsf{bad}_2\right] \leq \sum_{i=1}^{p} \frac{2(i-1)2^r}{2^n - p} \leq \sum_{i=1}^{p} \frac{4(i-1)}{2^c} \leq \frac{2p^2}{2^c},$$

Now, only the calculation of the probability of bad_3 remains. Define an event $\mathsf{NICE} = \neg(\mathsf{bad}_1 \vee \mathsf{bad}_2)$. Define the following graph $G = (V, E)$ based on τ_p:

$$V = \left\{ u^{(c)} \mid (u, \cdot) \in \tau_p \vee (\cdot, u) \in \tau_p \right\},$$
$$E = \left\{ (u^{(c)}, v^{(c)}, l) \mid (u, v) \in \tau_p \wedge l = (u^{(r)}, v^{(r)}) \right\}.$$

Claim 1: If NICE holds, then the graph G is a forest.
Define a subgraph $G_i = (V_i, E_i)$ as

$$V_i = \left\{ u_j^{(c)} \mid ((u_j, \cdot) \in \tau_p \vee (\cdot, u_j) \in \tau_p) \wedge 1 \leq j \leq i \right\},$$
$$E_i = \left\{ (u_j^{(c)}, v_j^{(c)}, l_j) \mid (u_j, v_j) \in \tau_p \wedge l_j = (u_j^{(r)}, v_j^{(r)}) \wedge 1 \leq j \leq i \right\}.$$

Define an event $F_i := G_i$ is a forest. Now we use a mathematical induction. As bad_1 does not happen, self-loop does not appear in G_1. Hence F_1 holds. Assume F_{i-1} holds. To make F_i false, $(u_i, v_i) \in \tau_p$ should create a self-loop or a cycle. But bad_1 does not happen, $u_i^{(c)} \neq v_i^{(c)}$. Hence (u_i, v_i) cannot make a self-loop. To make a cycle, at least $u_i^{(c)} \in V_{i-1} \wedge v_i^{(c)} \in V_{i-1}$ is needed that is exactly bad_2. But bad_2 does not happen, therefore, F_i holds.

Claim 2: For $1 \leq j \leq q$, $\Pr\left[\mathsf{bad}_3^j \wedge \mathsf{NICE}\right] \leq \frac{p}{2^\lambda}$.

For some ℓ^j, define a *potential chain* as any values $v_{j,1}, \ldots, v_{j,\ell^j}$ such that $v_{j,0} := S_{j,0}$ and satisfies a below condition:

- For all $1 \leq i \leq \ell^j$, $(v_{j,i-1} \oplus \mu_{j,i} \parallel 0^c, v_{j,i}) \in \tau_p$

with some values $\mu_{j,1}, \ldots, \mu_{j,\ell^j}$. If NICE holds, the graph $G = (V, E)$ is a forest by the Claim 1. Hence, $S_{j,0}^{(c)} \in V$ is an element of some tree T in G. As the tree has a unique path, at most p potential chains with a starting point $v_{j,0} = S_{j,0}^{(c)}$ can exist in G.

The event bad_3^j happens if and only if $I_{j,i} = \mu_{j,i}$ for all $1 \leq i \leq \ell_j$. Let $\mathcal{I}^j$ be a random variable about $I_{j,1} \parallel \cdots \parallel I_{j,\ell_j}$. Then, by the legitimacy condition of the adversary, the following inequality holds for any $\tau_j = (\tau_p, \tau_q \backslash Q_j)$ and any adversary's state $S_{all,j}$.

$$\mathsf{p}_{\mathsf{id}}\left[\mathsf{bad}_3^j \mid \tau_j \wedge S_{all,j} \wedge \mathsf{NICE}\right] \leq p \cdot \mathsf{Pred}\left(\mathcal{I}^j \mid \tau_j \wedge S_{all,j} \wedge \mathsf{NICE}\right).$$

Let Γ_j be the set of all possible tuple $(\tau_j, S_{all,j})$ and $g\Gamma_j$ be the set of all possible $(\tau_j, S_{all,j})$ that satisfies NICE. Then with the above inequality, we can obtain the following inequality.

$$\begin{aligned}
\mathsf{p_{id}}\left[\mathsf{bad}_3^j \wedge \mathsf{NICE}\right] &= \sum_{(\tau_j, S_{all,j}) \in \Gamma_j} \mathsf{p_{id}}\left[\tau_j \wedge S_{all,j}\right] \cdot \mathsf{p_{id}}\left[\mathsf{bad}_3^j \wedge \mathsf{NICE} | \tau_j \wedge S_{all,j}\right] \\
&= \sum_{(\tau_j, S_{all,j}) \in g\Gamma_j} \mathsf{p_{id}}\left[\tau_j \wedge S_{all,j}\right] \cdot \mathsf{p_{id}}\left[\mathsf{bad}_3^j | \tau_j \wedge S_{all,j}\right] \\
&\leq p \sum_{(\tau_j, S_{all,j}) \in g\Gamma_j} \mathsf{p_{id}}\left[\tau_j \wedge S_{all,j}\right] \cdot \mathsf{Pred}(\mathcal{I}^j | \tau_j \wedge S_{all,j}) \\
&\leq p \cdot \mathsf{Pred}(\mathcal{I}^j | \mathcal{T}_j \wedge \mathcal{S}_{all,j}) \leq \frac{p}{2^\lambda}
\end{aligned}$$

where $\mathcal{T}_j$ is a random variable about τ_j and $\mathcal{S}_{all,j}$ is a random variable about $S_{all,j}$. Now we can bound a probability of bad events.

$$\begin{aligned}
\mathsf{p_{id}}\left[\mathsf{bad}\right] &= \mathsf{p_{id}}\left[\neg\mathsf{NICE}\right] + \mathsf{p_{id}}\left[\mathsf{NICE} \wedge \left(\bigvee_{j=1}^{q} \mathsf{bad}_3^j\right)\right] \\
&\leq \mathsf{p_{id}}\left[\neg\mathsf{NICE}\right] + \sum_{j=1}^{q} \mathsf{p_{id}}\left[\mathsf{NICE} \wedge \mathsf{bad}_3^j\right] \\
&\leq \frac{2p(p+1)}{2^c} + \frac{pq}{2^\lambda}.
\end{aligned}$$

Good event analysis. For a good transcript $\tau = (\tau_p, \tau_q)$, we have

$$\begin{aligned}
\frac{\mathsf{p_{re}}\left[(\tau_p, \tau_q)\right]}{\mathsf{p_{id}}\left[(\tau_p, \tau_q)\right]} &= \frac{\Pr\left[\pi \vdash \tau_p\right] \Pr\left[\pi \vdash \tau_q \mid \pi \vdash \tau_p\right]}{\Pr\left[\pi \vdash \tau_p\right] \Pr\left[S_{1,\ell_1+w_1}, \ldots, S_{q,\ell_q+w_q}, y_1, \ldots, y_q\right]} \\
&= \frac{\Pr\left[\pi \vdash \tau_q \mid \pi \vdash \tau_p\right]}{1/2^{qn+r\sum_{i=1}^{q} w_i}} \\
&= 2^{qn+r\sum_{i=1}^{q} w_i} \Pr\left[\pi \vdash \tau_q \mid \pi \vdash \tau_p\right] \qquad (5)
\end{aligned}$$

Remark. Since the event bad_3 does not occur for $Q_j \in \tau_q$ (i.e., the potential chain is not fully connected with $I_{j,1}, \ldots, I_{j,\ell_j}$), the adversary does not have information of intermediate states $v_{j,\ell_j^c+1}, \ldots, v_{j,\ell_j+w_j-1}$:

1. $v_{j,i} = \pi(v_{j,i-1} \oplus I_{j,i} \parallel 0^c)$ for $\ell_j^c < i \leq \ell_j$,
2. $v_{j,i} = \pi(v_{j,i-1}) \oplus 0^r \parallel v_{j,i-1}^{(c)}$ for $\ell_j < i < \ell_j + w_j$.

Consequently, the intermediate states $v_{j,\ell_j^c+1}, \ldots, v_{j,\ell_j+w_j-1}$ should be treated as random variables for $1 \leq j \leq q$. To establish a lower bound for $\Pr\left[\pi \vdash \tau_q \mid \pi \vdash \tau_p\right]$, we need to count the valid assignments for these random variables. The assignment of specific values to the random variables $v_{t,\ell_t^c+1}, \ldots, v_{t,\ell_t+w_t-1}$ for $1 \leq t \leq j$ is represented by $\mathbf{S_j}$. To ensure that $\mathbf{S_j}$ is a valid assignment, we impose four conditions on $\mathbf{S_j}$. The reasons for each condition are as follows:

– Algorithms refresh and next generate state differently, necessitating the first two conditions on $\mathbf{S_j}$.
– For next, the upper r bits of the state are already known to the adversary from the output, requiring the third condition.
– In the M-REC oracle, the adversary knows the final state, which makes the last condition necessary.

Then the set τ_p^j is derived from processing $\mathbf{S_j}$ in the format of τ_p, while Γ_p^j denotes the collection of these sets that do not collide. We will now formally define the tuples and the sets.

For $1 \le j \le q$, $1 \le t \le j$, $\ell_t^c < i < \ell_t + w_t$ and any $s_{j,i} \in \{0,1\}^n$, let $\mathbf{S_j}$ be a tuple of $(s_{1,\ell_1^c+1}, \ldots, s_{1,\ell_1+w_1-1}, s_{2,\ell_2^c+1}, \ldots, s_{j,\ell_j+w_j-1})$ that satisfies the following conditions:

– $s_{t,i} = \pi(s_{t,i-1} \oplus I_{t,i} \,\|\, 0^c)$ for $\ell_t^c < i \le \ell_t$,
– $s_{t,i} = \pi(s_{t,i-1}) \oplus 0^r \,\|\, s_{t,i-1}^{(c)}$ for $\ell_t < i < \ell_t + w_t$,
– $s_{t,i}^{(r)} = y_t[r(i-\ell_t) : r(i-\ell_t+1)-1]$ for $\ell_t \le i \le \ell_t + w_t - 1$,
– $S_{t,\ell_t+w_t} = \pi(s_{t,\ell_t+w_t-1}) \oplus 0^r \,\|\, s_{t,\ell_t+w_t-1}^{(c)}$.

Note that, by the definition of ℓ_j^c, $(s_{t,\ell_j^c-1} \oplus I_{t,\ell_j^c} \,\|\, 0^c, s_{t,\ell_j^c}) \in \tau_p$. For each $\mathbf{S_j}$, we define a set τ_p^j as follows:

$$\begin{aligned}\tau_p^j := \tau_p &\cup \{(s_{t,i-1} \oplus I_{t,i} \,\|\, 0^c, s_{t,i}) \mid 1 \le t \le j, \ell_t^c < i \le \ell_t\} \\ &\cup \left\{(s_{t,i-1}, s_{t,i} \oplus 0^r \,\|\, s_{t,i-1}^{(c)} \mid 1 \le t \le j, \ell_t < i < \ell_t + w_t\right\} \\ &\cup \left\{(s_{t,\ell_t+w_t-1}, S_{t,\ell_t+w_t} \oplus 0^r \,\|\, s_{t,\ell_t+w_t-1}^{(c)}) \mid 1 \le t \le j\right\}.\end{aligned}$$

Let Γ_p^j be a set of all possible τ_p^j that satisfies the following condition:

– For every distinct $(u,v),(u',v') \in \tau_p^j$, $u \ne u'$ and $v \ne v'$

Note that for any $\tau_p^j \in \Gamma_p^j$ and $\pi \vdash \tau_p^j$, $\pi \vdash (\tau_p, Q_1, \ldots, Q_j)$ holds. Let $\pi \vdash \Gamma_p^j$ be an event that there exists $\tau_p^j \in \Gamma_p^j$ that satisfies $\pi \vdash \tau_p^j$ by slightly abusing notation. Define $\Gamma_p^0 := \{\tau_p\}$. Then we obtain the following inequalities.

$$\begin{aligned}\Pr\left[\pi \vdash \tau_q \mid \pi \vdash \tau_p\right] &\ge \Pr\left[\pi \vdash \Gamma_p^q \mid \pi \vdash \tau_p\right] \\ &\ge \prod_{j=1}^{q} \Pr\left[\pi \vdash \Gamma_p^j \mid \pi \vdash \Gamma_p^{j-1}\right]. \end{aligned} \tag{6}$$

The first inequality follows from the additional conditions imposed on the definition of Γ_p^q. The last inequality holds since $\pi \vdash \Gamma_p^j \wedge \pi \vdash \Gamma_p^{j-1}$ is the same as $\pi \vdash \Gamma_p^j$.

Now, we should calculate $\Pr\left[\pi \vdash \Gamma_p^j \mid \pi \vdash \Gamma_p^{j-1}\right]$ for $1 \le j \le q$. For $1 \le j \le q$ and $\ell_j^c < i < \ell_j + w_j$, we recursively define a random variable $\mathcal{S}_{j,i}$ as follows.

– $\mathcal{S}_{j,i} := \pi(\mathcal{S}_{j,i-1} \oplus I_{j,i} \,\|\, 0^c)$ for $\ell_j^c < i \le \ell_j$

– $\mathcal{S}_{j,i} := \pi(\mathcal{S}_{j,i-1}) \oplus 0^r \parallel \mathcal{S}^{(c)}_{j,i-1}$ for $\ell_j < i \leq \ell_j + w_j - 1$

where $\mathcal{S}_{j,\ell^c_j} = s_{j,\ell^c_j}$ and is already known to the adversary by the definition of ℓ^c_j.

Remark. We define the events FRESH^j_i to indicate that, for the j-th M-REC query, the input $\mathcal{S}_{j,i-1} \oplus I_{j,i} \parallel 0^c$ to π has not appeared in any prior query. Additionally, for $\ell_j < i \leq \ell_j + w_j$, we require that the upper r bits of $\mathcal{S}_{j,i-1}$ align with the output segment $y_j[r(i-\ell_j-1) : r(i-\ell_j)-1]$. By conditioning on FRESH^j_i, we can assume the presence of randomness when computing $\pi(\mathcal{S}_{j,i-1} \oplus I_{j,i} \parallel 0^c)$, where $I_{j,i} = 0^r$ for $\ell_j < i \leq \ell_j + w_j$.

Furthermore, since the adversary knows the final state S_{j,ℓ_j+w_j}, we introduce the event $\mathsf{FREE}^j_{\ell_j+w_j}$. This event ensures that the output $S_{j,\ell_j+w_j} \oplus 0^r \| \mathcal{S}^{(c)}_{j,\ell_j+w_j-1}$ of π has not appeared previously. We now formally define these events.

For any $\tau^{j-1}_p \in \Gamma^{j-1}_p$, define the following events:

– For every $\ell^c_j < i \leq \ell_j + w_j$, $\mathsf{FRESH}^{\mathsf{j}}_{\mathsf{i}}$: following events hold.
  - $(\mathcal{S}_{j,i-1} \oplus I_{j,i} \parallel 0^c, \cdot) \notin \tau^{j-1}_p$
  - $\mathcal{S}_{j,i-1} \oplus I_{j,i} \parallel 0^c \neq \mathcal{S}_{j,k-1} \oplus I_{j,k} \parallel 0^c$ for all $1 \leq k < i$
  - $\mathcal{S}^{(r)}_{j,i-1} = y_j[r(i-\ell_j-1) : r(i-\ell_j)-1]$ if $\ell_j < i \leq \ell_j + w_j$
– $\mathsf{FREE}^{\mathsf{j}}_{\ell_{\mathsf{j}}+\mathsf{w_j}}$: following events hold.
  - $(\cdot, S_{j,\ell_j+w_j} \oplus 0^r \parallel \mathcal{S}^{(c)}_{j,\ell_j+w_j-1}) \notin \tau^{j-1}_p$
  - $S_{j,\ell_j+w_j} \oplus 0^r \parallel \mathcal{S}^{(c)}_{j,\ell_j+w_j-1} \neq \mathcal{S}_{j,k}$ for all $\ell^c_j < k < \ell_j$
  - $S_{j,\ell_j+w_j} \oplus 0^r \parallel \mathcal{S}^{(c)}_{j,\ell_j+w_j-1} \neq \mathcal{S}_{j,k} \oplus 0^r \parallel \mathcal{S}^{(c)}_{j,k-1}$ for all $\ell_j \leq k < \ell_j + w_j - 1$
– $\mathsf{FRESH}^{\mathsf{j}} := \mathsf{FREE}^{\mathsf{j}}_{\ell_{\mathsf{j}}+\mathsf{w_j}} \wedge \left(\bigwedge_{t=\ell^c_j+1}^{\ell_j+w_j} \mathsf{FRESH}^{\mathsf{j}}_{\mathsf{t}} \right)$

We assume $p + \sigma \leq 2^{c-1}$. Then the following inequalities hold for $1 \leq j \leq q$ and any $\tau^{j-1}_p \in \Gamma^j_p$:

– For $\ell^c_j < i \leq \ell_j$,

$$\Pr\left[\neg\mathsf{FRESH}^{\mathsf{j}}_{\mathsf{i}} \mid \pi \vdash \tau^{j-1}_p \wedge \left(\bigwedge_{t=\ell^c_j+1}^{i-1} \mathsf{FRESH}^{\mathsf{j}}_{\mathsf{t}} \right)\right] \leq \frac{p+\sigma}{2^n - (p+\sigma)} \leq \frac{2(p+\sigma)}{2^n}.$$

In this case, it is necessary to select an appropriate $\mathcal{S}_{j,i-1}$ that corresponds to the output of $\pi(\mathcal{S}_{j,i-2} \oplus I_{j,i-1} \parallel 0^c)$. Given the conditions in the probability analysis, and considering FRESH^j_{i-1}, $\mathcal{S}_{j,i-1}$ must not collide with any previously observed outputs of π. Since there are at least $2^n - (p+\sigma)$ fresh outputs available, this quantity serves as the denominator in the probability calculation. To trigger the event $\neg\mathsf{FRESH}^j_i$, there are at most $p+\sigma$ non-fresh inputs to π, which are used as the numerator.

– For $\ell_j < i < \ell_j + w_j$,

$$\begin{aligned}
&\Pr\left[\neg\mathsf{FRESH}^{\mathsf{j}}_{\mathsf{i}} \mid \pi \vdash \tau_p^{j-1} \wedge \left(\bigwedge_{t=\ell_j^c+1}^{i-1} \mathsf{FRESH}^{\mathsf{j}}_{\mathsf{t}}\right)\right] \\
&\leq \frac{p+\sigma}{2^n-(p+\sigma)} + \frac{(2^r-1)2^c}{2^n-(p+\sigma)} = \frac{p+\sigma}{2^n-(p+\sigma)} + \left(1 - \frac{2^c-(p+\sigma)}{2^n-(p+\sigma)}\right) \\
&\leq \frac{2(p+\sigma)}{2^n} + \left(1 - \frac{2^c-(p+\sigma)}{2^n}\right) \leq 1 - \frac{1}{2^r}\left(1 - \frac{3(p+\sigma)}{2^c}\right)
\end{aligned}$$

For $\ell_j < i \leq \ell_j + w_j$, we must account for the final condition on FRESH^j_i. The term $\frac{p+\sigma}{2^n-(p+\sigma)}$ corresponds to the first two conditions of FRESH^j_i, derived using the same reasoning as in the previous case. Meanwhile, the term $\frac{(2^r-1)2^c}{2^n-(p+\sigma)}$ arises from the last condition. Without considering the specific value of $y_j[r(i-\ell_j-1) : r(i-\ell_j)-1]$, there are at most $(2^r-1)2^c$ possible choices for $\mathcal{S}_{j,i-1}$ violating the last condition.

–

$$\begin{aligned}
&\Pr\left[\neg\left(\mathsf{FRESH}^{\mathsf{j}}_{\ell_{\mathsf{j}}+\mathsf{w_j}} \wedge \mathsf{FREE}^{\mathsf{j}}_{\ell_{\mathsf{j}}+\mathsf{w_j}}\right) \mid \pi \vdash \tau_{\mathsf{p}}^{\mathsf{j}-1} \wedge \left(\bigwedge_{\mathsf{t}=\ell_{\mathsf{j}}^{\mathsf{c}}+1}^{\ell_{\mathsf{j}}+\mathsf{w_j}-1} \mathsf{FRESH}^{\mathsf{j}}_{\mathsf{t}}\right)\right] \\
&\leq \frac{2(p+\sigma)}{2^n-(p+\sigma)} + \left(1 - \frac{2^c-(p+\sigma)}{2^n}\right) \leq 1 - \frac{1}{2^r}\left(1 - \frac{5(p+\sigma)}{2^c}\right)
\end{aligned}$$

We obtain $\frac{p+\sigma}{2^n-(p+\sigma)} + \left(1 - \frac{2^c-(p+\sigma)}{2^n}\right)$ for the $\neg\mathsf{FRESH}^j_{\ell_j+w_j}$ event, following the same reasoning as in the previous case. Similarly, for the $\neg\mathsf{FREE}^j_{\ell_j+w_j}$ event, we have $\frac{p+\sigma}{2^n-(p+\sigma)}$, derived using an approach analogous to the first case. Applying the union bound completes the calculation.

–

$$\Pr\left[\pi(\mathcal{S}_{j,\ell_j+w_j-1}) = S_{j,\ell_j+w_j} \oplus 0^r \parallel \mathcal{S}^{(c)}_{j,\ell_j+w_j-1} \mid \pi \vdash \tau_p^{j-1} \wedge \mathsf{FRESH}^{\mathsf{j}}\right] \geq \frac{1}{2^n}$$

Conditioned on FRESH^j, selecting $\pi(\mathcal{S}_{j,\ell_j+w_j-1})$ does not lead to any contradictions. Therefore, we can assume that $\pi(\mathcal{S}_{j,\ell_j+w_j-1})$ behaves as a uniformly random value under this condition. Consequently, the probability is at least $1/2^n$.

Note that $p+\sigma \le 2^{c-1}$. Using the above inequalities, for any $\tau_p^{j-1} \in \Gamma_p^{j-1}$, we can bound the following probabilities.

$$\begin{aligned}
&\Pr\left[\pi \vdash \Gamma_p^j \mid \pi \vdash \tau_p^{j-1}\right] \\
&\ge \Pr\left[\pi \vdash \Gamma_p^j \mid \pi \vdash \tau_p^{j-1} \wedge \mathsf{FRESH}^{\mathsf{j}}\right] \cdot \Pr\left[\mathsf{FRESH}^{\mathsf{j}} \mid \pi \vdash \tau_p^{j-1}\right] \\
&\ge \Pr\left[\pi(\mathcal{S}_{j,\ell_j+w_j-1}) = S_{j,\ell_j+w_j} \oplus 0^r \parallel \mathcal{S}^{(c)}_{j,\ell_j+w_j-1} \mid \pi \vdash \tau_p^{j-1} \wedge \mathsf{FRESH}^{\mathsf{j}}\right] \\
&\cdot \Pr\left[\mathsf{FRESH}^{\mathsf{j}} \mid \pi \vdash \tau_p^{j-1}\right] \\
&\ge \frac{1}{2^{n+rw_j}} \left(1 - \frac{2(p+\sigma)}{2^n}\right)^{\ell_j - \ell_j^c} \left(1 - \frac{3(p+\sigma)}{2^c}\right)^{w_j - 1} \left(1 - \frac{5(p+\sigma)}{2^c}\right). \qquad (7)
\end{aligned}$$

Given $\mathsf{FRESH}^{\mathsf{j}}$, the condition $\tau \vdash \Gamma_p^j$ holds if the event $\pi(\mathcal{S}_{j,\ell_j+w_j-1}) = S_{j,\ell_j+w_j} \oplus 0^r \parallel \mathcal{S}^{(c)}_{j,\ell_j+w_j-1}$ occurs. Thus, the penultimate inequality holds.

For simple notation, we will denote the last concrete bound of the (7) as B:

$$\mathsf{B} := \frac{1}{2^{n+rw_j}} \left(1 - \frac{2(p+\sigma)}{2^n}\right)^{\ell_j - \ell_j^c} \left(1 - \frac{3(p+\sigma)}{2^c}\right)^{w_j - 1} \left(1 - \frac{5(p+\sigma)}{2^c}\right).$$

Note that, for any distinct $\tau_1, \tau_2 \in \Gamma_p^{j-1}$, $\pi \vdash \tau_1$ and $\pi \vdash \tau_2$ are mutually exclusive. Therefore, we can obtain the following using B.

$$\begin{aligned}
&\Pr\left[\pi \vdash \Gamma_p^j \mid \pi \vdash \Gamma_p^{j-1}\right] = \frac{\Pr\left[\pi \vdash \Gamma_p^j \wedge \pi \vdash \Gamma_p^{j-1}\right]}{\Pr\left[\pi \vdash \Gamma_p^{j-1}\right]} = \frac{\Pr\left[\pi \vdash \Gamma_p^j\right]}{\Pr\left[\pi \vdash \Gamma_p^{j-1}\right]} \\
&= \frac{\sum_{\pi_p^{j-1} \in \Gamma_p^{j-1}} \Pr\left[\pi \vdash \Gamma_p^j \wedge \pi \vdash \tau_p^{j-1}\right]}{\sum_{\pi_p^{j-1} \in \Gamma_p^{j-1}} \Pr\left[\pi \vdash \tau_p^{j-1}\right]} \\
&= \frac{\sum_{\pi_p^{j-1} \in \Gamma_p^{j-1}} \Pr\left[\pi \vdash \Gamma_p^j \mid \pi \vdash \tau_p^{j-1}\right] \cdot \Pr\left[\pi \vdash \tau_p^{j-1}\right]}{\sum_{\pi_p^{j-1} \in \Gamma_p^{j-1}} \Pr\left[\pi \vdash \tau_p^{j-1}\right]} \\
&\ge \frac{\mathsf{B} \cdot \sum_{\pi_p^{j-1} \in \Gamma_p^{j-1}} \Pr\left[\pi \vdash \tau_p^{j-1}\right]}{\sum_{\pi_p^{j-1} \in \Gamma_p^{j-1}} \Pr\left[\pi \vdash \tau_p^{j-1}\right]} = \mathsf{B} \qquad (8)
\end{aligned}$$

Hence, by (6) and (8), the following holds,

$$\begin{aligned}
&\Pr\left[\pi \vdash \tau_q \mid \pi \vdash \tau_p\right] \\
&\ge \frac{1}{2^{qn+r\sum_{j=1}^q w_j}} \left(1 - \frac{2(p+\sigma)}{2^n}\right)^{\sum_{j=1}^q \ell_j - \ell_j^c} \left(1 - \frac{5(p+\sigma)}{2^c}\right)^{\sum_{j=1}^q w_j} \\
&\ge \frac{1}{2^{qn+r\sum_{j=1}^q w_j}} \left(1 - \frac{5\sigma(p+\sigma)}{2^c}\right).
\end{aligned}$$

Then, with (5) and the above inequality, we have

$$\frac{\mathsf{p}_{\mathsf{re}}\left[(\tau_p, \tau_q)\right]}{\mathsf{p}_{\mathsf{id}}\left[(\tau_p, \tau_q)\right]} \ge 1 - \frac{5\sigma(p+\sigma)}{2^c}.$$

By using the H-Coefficient technique, we can conclude the proof.

4.2 Proof of (3)

Overview. Unlike in the M-Rec game, the initial state $S_{i,0}$ in the M-Pres game is uniformly random. Consequently, no potential chain can originate from the initial state within the birthday bound with respect to c by defining appropriate bad cases. The remaining probability analysis follows from the FRESH and FREE techniques used in the proof of (2).

Proof. In this section, we will use $\sigma := \lceil \frac{n}{r} \rceil (\sigma_1 + \sigma_2)$. Suppose that at the end of the game, we give the following values to the adversary. Note that giving the values only increases the adversary's advantage.

- DRBG state immediately after INIT or INTER queries.
  1. If an adversary makes $\mathrm{ROR}_{\mathrm{mpres}}$ exactly after the INIT or INTER query, then only the lower c-bit of the state is given to the adversary. This is because an adversary can obtain the upper r-bit of the state by the $\mathrm{ROR}_{\mathrm{mpres}}$. Hence, an adversary can fully recover the $n = r + c$-bit state.
  2. Otherwise, the full n bit state is given.

  The given state will be denoted $S_{i,0}$ in the transcript τ.
- DRBG state immediately after the last $\mathrm{ROR}_{\mathrm{mpres}}$ query before each INTER query and all states between that $\mathrm{ROR}_{\mathrm{mpres}}$ and the INTER(= all query results for π starting from the DRBG state after the last $\mathrm{ROR}_{\mathrm{mpres}}$ query). Note that if an adversary does not make $\mathrm{ROR}_{\mathrm{mpres}}$ query, then we give every state between two INTER queries.
  In the transcript τ,
  1. S_{i,ℓ'_i}:= DRBG state immediately after the last $\mathrm{ROR}_{\mathrm{mpres}}$ query,
  2. $\{(u_i, v_i)\}_{i=p+1,\ldots,p'}$ (within the transcript τ_p):= Using the given states, we can derive the input-output relations of π. Accordingly, we present these relations in the set.
- For each $\mathrm{ROR}_{\mathrm{mpres}}$ call, if the output length is not a multiple of r, the truncated part is provided to the adversary. In the ideal world, it is generated randomly and provided.

Then the transcript can be constructed as

$$\tau = (\tau_p, \tau_q),$$

where,

- τ_p is the set of query results for the permutation π, represented as (u, v) pairs where $\pi(u) = v$. We define $\tau_p = \{(u_i, v_i)\}_{i=1,\ldots,p'}$, where $p' - p$ denotes the number of additional permutation results provided at the end of the game.
  1. $\{(u_i, v_i)\}_{i=1,\ldots,p}$ represents the primitive queries made by the adversary.
  2. $\{(u_i, v_i)\}_{i=p+1,\ldots,p'}$ represents the input-output relations of π that are provided additionally at the end of the game.

  Note that $p' \leq p + \sigma$.
- $\tau_q = (\mathcal{Q}_1, \cdots, \mathcal{Q}_q)$.

– $\mathcal{Q}_i$ is a tuple representing the query history between the $(i-1)$-th INTER and the i-th INTER query (where INIT can be considered as the 0-th INTER). The elements of $\mathcal{Q}_i$ are given as:

$$(S_{i,0}, Z_{i,1}, \cdots, Z_{i,\ell'_i}, S_{i,\ell'_i}, b_{i,1}, \cdots, b_{i,\ell'_i})$$

where:

- $S_{i,0}$ is the state of the DRBG immediately after the $(i-1)$-th INTER query.
- ℓ'_i denotes the number of invocations of π between the $(i-1)$-th INTER query and the last $\mathrm{ROR}_{\mathrm{mpres}}$ query preceding the i-th INTER query.
- $b_{i,j} \in \{0,1\}$ is the indicator for the j-th invocation of π after the $(i-1)$-th INTER. If $b_{i,j} = 1$, π is used in $\mathrm{REF}_{\mathrm{mpres}}$, and if $b_{i,j} = 0$, π is used in $\mathrm{ROR}_{\mathrm{mpres}}$.
- $Z_{i,j}$ represents either the entropy input of $\mathrm{REF}_{\mathrm{mpres}}$ or part of the $\mathrm{ROR}_{\mathrm{mpres}}$ output queried after the $(i-1)$-th INTER. Specifically, if $b_{i,j} = 1$, $Z_{i,j}$ is the input to $\mathrm{REF}_{\mathrm{mpres}}$, and if $b_{i,j} = 0$, $Z_{i,j}$ is part of the output of $\mathrm{ROR}_{\mathrm{mpres}}$.
- S_{i,ℓ'_i} is the state of the DRBG immediately after the last $\mathrm{ROR}_{\mathrm{mpres}}$ query, before the i-th INTER query.

Without loss of generality, suppose for some q' with $1 \le q' \le q$, $\mathcal{Q}_i$ with $1 \le i \le q'$ has $\ell'_i = 1$ and $b_{i,1} = 0$, $\mathcal{Q}_j$ with $q'+1 \le j \le q$ doesn't. Then we can rewrite the transcript as

$$\tau = (\tau_p, \tau_d, \tau_u),$$

where $\tau_d = (\mathcal{Q}_1, \cdots, \mathcal{Q}_{q'})$ and $\tau_u = (\mathcal{Q}_{q'+1}, \cdots, \mathcal{Q}_q)$.

Bad event analysis. We define bad events for M-Pres game as follows. We use the bitwise AND operation (&) to handle both cases of $b_{i,1} = 0$ and $b_{i,1} = 1$ simultaneously.

– bad_1 : For any $1 \le i \le q$, there exists $(u, \cdot) \in \tau_p$ such that $S_{i,0} \oplus b^r_{i,1} \& Z_{i,1} \| 0^c = u$.
– bad'_1 : For any $1 \le i \ne j \le q$, $S_{i,0} \oplus b^r_{i,1} \& Z_{i,1} \parallel 0^c = S_{j,0} \oplus b^r_{j,1} \& Z_{j,1} \parallel 0^c$ holds.
– bad_2 : For any $1 \le i \le q'$, there exists $(\cdot, v) \in \tau_p$ such that $S_{i,1} \oplus 0^r \parallel S^{(c)}_{i,0} = v$.
– bad'_2 : For any $1 \le i \ne j \le q'$, $S_{i,1} \oplus 0^r \parallel S^{(c)}_{i,0} = S_{j,1} \oplus 0^r \parallel S^{(c)}_{j,0}$ holds.

Then we have

$$\mathsf{p}_{\mathsf{id}}[\mathsf{bad}_1] \le \frac{(p+\sigma)q}{2^c}, \qquad \mathsf{p}_{\mathsf{id}}[\mathsf{bad}'_1] \le \frac{0.5q^2}{2^c},$$
$$\mathsf{p}_{\mathsf{id}}[\mathsf{bad}_2] \le \frac{(p+\sigma)q'}{2^c}, \qquad \mathsf{p}_{\mathsf{id}}[\mathsf{bad}'_2] \le \frac{0.5q'^2}{2^c}.$$

Remark. Unlike the M-Rec game, in the M-Pres game, the initial state $S_{i,0}$ is chosen uniformly at random. Therefore, by excluding the event bad_1, a potential

chain cannot even begin from the initial state. Additionally, by excluding the event bad'_1, we can assume that the initial states after INTER queries are all distinct.

In the case of τ_d, the adversary learns the upper r bits of $S_{i,0}$ from the $\mathrm{ROR}_{\mathrm{mpres}}$ output and also learns the resulting state $S_{i,1}$ during the interaction. Thus, the only information unknown to the adversary is the lower c bits of $S_{i,0}$ (note that this will be revealed after the interaction). Consequently, by excluding the event bad_2, the adversary cannot directly recover the full $S_{i,0}$ using primitive queries before the interaction ends. Furthermore, by excluding the event bad'_2, collisions in the outputs of π are avoided.

Good event analysis. For a good transcript $\tau = (\tau_p, \tau_d, \tau_u)$, we have

$$\begin{aligned}
&\frac{\mathsf{p}_{\mathsf{re}}\left[(\tau_p,\tau_d,\tau_u)\right]}{\mathsf{p}_{\mathsf{id}}\left[(\tau_p,\tau_d,\tau_u)\right]} \\
&= \frac{\Pr\left[\pi \vdash \tau_p\right]\Pr\left[\bigwedge_{j=1}^{q} S_{j,0}\right]\Pr\left[\pi \vdash \tau_d \mid \pi \vdash \tau_p\right]\Pr\left[\pi \vdash \tau_u \mid \pi \vdash (\tau_p,\tau_d)\right]}{\Pr\left[\pi \vdash \tau_p\right]\Pr\left[\bigwedge_{j=1}^{q} S_{j,0}\right]\Pr\left[\tau_d\right]\prod_{i=q'+1}^{q}\Pr\left[\mathcal{Q}_i\right]} \\
&= \frac{\Pr\left[\pi \vdash \tau_d \mid \pi \vdash \tau_p\right]}{1/2^{q'n}} \cdot \frac{\Pr\left[\pi \vdash \tau_u \mid \pi \vdash (\tau_p,\tau_d)\right]}{\prod_{i=q'+1}^{q} 1/2^{n} \cdot 1/2^{r\ell''_i}} \\
&= 2^{qn+r\sum_{i=q'+1}^{q}\ell''_i}\Pr\left[\pi \vdash \tau_d \mid \pi \vdash \tau_p\right] \cdot \Pr\left[\pi \vdash \tau_u \mid \pi \vdash (\tau_p,\tau_d)\right]. \qquad (9)
\end{aligned}$$

where ℓ''_i is the number of $b_{i,j}$ with $b_{i,j} = 0$ for $2 \le j \le \ell'_i$.

Claim 1 : $\Pr\left[\pi \vdash \tau_d \mid \pi \vdash \tau_p\right] \ge \frac{1}{2^{q'n}}$.

Since any bad events do not happen, for $1 \le i \le q'$,

$$\Pr\left[\pi(S_{i,0}) = S_{i,1} \oplus 0^r \parallel S^{(c)}_{i,0} \mid \pi \vdash (\tau_p, \mathcal{Q}_1, \cdots, \mathcal{Q}_{i-1})\right] \ge \frac{1}{2^n}.$$

Claim 2 : $\Pr\left[\pi \vdash \tau_u \mid \pi \vdash (\tau_p,\tau_d)\right] \ge \frac{1}{2^{(q-q')n+r\sum_{i=q'+1}^{q}\ell''_i}}\left(1 - \frac{6\sigma(p+\sigma)}{2^c}\right)$.

Let $\tau_p^{q'} := \tau_p \cup \{(S_{1,0}, S_{1,1} \oplus 0^r \parallel S^{(c)}_{1,0}), \cdots, (S_{q',0}, S_{q',1} \oplus 0^r \parallel S^{(c)}_{q',0})\}$ then it is trivial that $\pi \vdash \tau_p^{q'} \Leftrightarrow \pi \vdash (\tau_p, \tau_d)$.

For $q'+1 \le j \le q$, $q'+1 \le t \le j$, $1 \le i \le \ell'_t$, and any $s_{t,i} \in \{0,1\}^n$, let $\mathbf{S_j}$ be a tuple of $(s_{q'+1,1}, s_{q'+1,2}, \ldots, s_{q'+1,\ell'_{q'+1}-1}, s_{q'+2,1}, \ldots, s_{j,\ell'_j-1})$ that satisfies

- for all $1 \le i \le \ell'_t$,

$$\pi(s_{t,i-1} \oplus b^r_{t,i} \& Z_{t,i} \parallel 0^c) = s_{t,i} \oplus 0^r \parallel \left((1^c \oplus b^c_{t,i}) \& s^{(c)}_{t,i-1}\right)$$

  where $s_{t,0} := S_{t,0}$ and $s_{t,\ell'_t} := S_{t,\ell'_t}$.
- for all $1 \le i \le \ell'_t$, if $b_{t,i} = 0$ holds,

$$s^{(r)}_{t,i-1} = Z_{t,i}.$$

  where $s_{t,0} := S_{t,0}$ and $s_{t,\ell'_t} := S_{t,\ell'_t}$.

For each $\mathbf{S}_j$, we define a set τ_p^j as follows:

$$\tau_p^j := \tau_p^{q'} \cup \{(X_{t,i}, Y_{t,i}) \mid q'+1 \le t \le j, 1 \le i \le \ell'_t\}.$$

where $X_{t,i} := s_{t,i-1} \oplus b_{t,i}^r \& Z_{t,i} \parallel 0^c$, $Y_{t,i} := s_{t,i} \oplus 0^r \parallel \left((1^c \oplus b_{t,i}^c) \& s_{t,i-1}^{(c)}\right)$, and $s_{t,0} := S_{t,0}$ and $s_{t,\ell'_t} := S_{t,\ell'_t}$. Let Γ_p^j be a set of all possible τ_p^j that satisfies the following condition:

- For every distinct $(u,v), (u',v') \in \tau_p^j$, $u \neq u'$ and $v \neq v'$.
- For every $(u, \cdot) \in \tau_p^j$, $S_{i,0} \oplus b_{i,1}^r \& Z_{i,1} \parallel 0^c \neq u$ for $j < i \le q$.

Note that for any $\tau_p^j \in \Gamma_p^j$ and $\pi \vdash \tau_p^j$, $\pi \vdash (\tau_p, Q_1, \ldots, Q_j)$ holds. Let $\pi \vdash \Gamma_p^j$ be an event that there exists $\tau_p^j \in \Gamma_p^j$ which satisfies $\pi \vdash \tau_p^j$ by slightly abusing notation. Define $\Gamma_p^{q'} = \tau_p^{q'}$. Then we obtain the following inequalities.

$$\begin{aligned}
\Pr\left[\pi \vdash \tau_u \mid \pi \vdash (\tau_p, \tau_d)\right] &\ge \Pr\left[\pi \vdash \tau_u \mid \pi \vdash \tau_p^{q'}\right] \\
&\ge \Pr\left[\pi \vdash \Gamma_p^q \mid \pi \vdash \tau_p^{q'}\right] \\
&\ge \prod_{j=q'+1}^{q} \Pr\left[\pi \vdash \Gamma_p^j \mid \pi \vdash \Gamma_p^{j-1}\right]. \qquad (10)
\end{aligned}$$

For $q'+1 \le j \le q$, and $1 \le i \le \ell'_j$, we recursively define a random variable $\mathcal{S}_{j,i}$ as

$$\pi(\mathcal{S}_{j,i-1} \oplus b_{j,i}^r \& Z_{j,i} \parallel 0^c) = \mathcal{S}_{j,i} \oplus 0^r \parallel \left((1^c \oplus b_{j,i}^c) \& \mathcal{S}_{j,i-1}^{(c)}\right)$$

where $\mathcal{S}_{j,0} = S_{j,0}$ and $\mathcal{S}_{j,\ell'_j} = S_{j,\ell'_j}$. For any $\tau_p^{j-1} \in \Gamma_p^{j-1}$, define the following events:

- For $1 \le i \le \ell'_j$, FRESH_i^j : following 3 events holds.
  - $\left(\mathcal{S}_{j,i-1} \oplus b_{j,i}^r \& Z_{j,i} \parallel 0^c, \cdot\right) \notin \tau_p^{j-1}$.
  - $\mathcal{S}_{j,i-1} \oplus b_{j,i}^r \& Z_{j,i} \parallel 0^c \neq \mathcal{S}_{j,k-1} \oplus b_{j,k}^r \& Z_{j,k} \parallel 0^c$ for all $1 \le k < i$.
  - If $b_{j,i} = 0$, $\mathcal{S}_{j,i-1}^{(r)} = Z_{j,i}$
- $\mathsf{FREE}_{\ell'_j}^j$: the following 2 events hold.
  - $\left(\cdot, S_{j,\ell'_j} \oplus 0^r \parallel \mathcal{S}_{j,\ell'_j-1}^{(c)}\right) \notin \tau_p^{j-1}$.
  - $S_{j,\ell'_j} \oplus 0^r \parallel \mathcal{S}_{j,\ell'_j-1}^{(c)} \neq \mathcal{S}_{j,i} \oplus 0^r \parallel \left((1^c \oplus b_{j,i}^c) \& \mathcal{S}_{j,i-1}^{(c)}\right)$ for all $1 \le i < \ell'_j$
- $\mathsf{FRESH}^j = \mathsf{FREE}_{\ell'_j}^j \wedge \left(\bigwedge_{i=1}^{\ell'_j} \mathsf{FRESH}_i^j\right)$.

Note that $\Pr\left[\mathsf{FRESH}_1^j \mid \pi \vdash \tau_p^{j-1}\right] = 1$ for every $q'+1 \le j \le q$. Then the following inequalities hold for $q'+1 \le j \le q$ and any $\tau_p^{j-1} \in \Gamma_p^{j-1}$:

– For $1 \le i < \ell'_j$:

$$\Pr\left[\mathsf{FRESH}^{\mathsf{j}}_{\mathsf{i}} \mid \pi \vdash \tau_p^{j-1} \wedge b_{j,i} = 1 \wedge \left(\bigwedge_{k=1}^{i-1} \mathsf{FRESH}^{\mathsf{j}}_{\mathsf{k}}\right)\right] \ge \frac{2^n - 2(p+\sigma)}{2^n},$$
$$\Pr\left[\mathsf{FRESH}^{\mathsf{j}}_{\mathsf{i}} \mid \pi \vdash \tau_p^{j-1} \wedge b_{j,i} = 0 \wedge \left(\bigwedge_{k=1}^{i-1} \mathsf{FRESH}^{\mathsf{j}}_{\mathsf{k}}\right)\right] \ge \frac{2^c - 4(p+\sigma)}{2^n}.$$

–

$$\Pr\left[\mathsf{FRESH}^{\mathsf{j}}_{\ell'_{\mathsf{j}}} \wedge \mathsf{FREE}^{j}_{\ell'_j} \mid \pi \vdash \tau_p^{j-1} \wedge b_{j,\ell'_j} = 0 \wedge \left(\bigwedge_{k=1}^{\ell'_j-1} \mathsf{FRESH}^{\mathsf{j}}_{\mathsf{k}}\right)\right] \ge \frac{2^c - 6(p+\sigma)}{2^n}.$$

Note that b_{j,ℓ'_j} is always 0 by the definition of ℓ'_j.

–

$$\Pr\left[\pi(\mathcal{S}_{j,\ell'_j-1}) = S_{j,\ell'_j} \oplus 0^r \parallel \mathcal{S}^{(c)}_{j,\ell'_j-1} \mid \pi \vdash \tau_p^{j-1} \wedge \mathsf{FRESH}^{\mathsf{j}}\right] \ge \frac{1}{2^n}.$$

Using the above inequalities, for any $\tau_p^{j-1} \in \Gamma_p^{j-1}$, we can bound the below probabilities.

$$\begin{aligned}
&\Pr\left[\pi \vdash \Gamma_p^j \mid \pi \vdash \tau_p^{j-1}\right] \\
&\ge \Pr\left[\pi \vdash \Gamma_p^j \mid \pi \vdash \tau_p^{j-1} \wedge \mathsf{FRESH}^{\mathsf{j}}\right] \cdot \Pr\left[\mathsf{FRESH}^{\mathsf{j}} \mid \pi \vdash \tau_p^{j-1}\right] \\
&\ge \Pr\left[\pi(\mathcal{S}_{j,\ell'_j-1}) = S_{j,\ell'_j} \oplus 0^r \parallel \mathcal{S}_{j,\ell'_j-1} \mid \pi \vdash \tau_p^{j-1} \wedge \mathsf{FRESH}^{\mathsf{j}}\right] \\
&\cdot \Pr\left[\mathsf{FRESH}^{\mathsf{j}} \mid \pi \vdash \tau_p^{j-1}\right] \\
&\ge \frac{1}{2^n} \Pr\left[\mathsf{FRESH}^{\mathsf{j}} \mid \pi \vdash \tau_p^{j-1}\right] \\
&\ge \frac{1}{2^n} \left(\frac{2^n - 2(p+\sigma)}{2^n}\right)^{\ell'_j - \ell''_j} \left(\frac{2^c - 4(p+\sigma)}{2^n}\right)^{\ell''_j - 1} \frac{2^c - 6(p+\sigma)}{2^n} \\
&\ge \frac{1}{2^{n+r\ell''_j}} \left(1 - \frac{2(p+\sigma)}{2^n}\right)^{\ell'_j - \ell''_j} \left(1 - \frac{4(p+\sigma)}{2^c}\right)^{\ell''_j - 1} \left(1 - \frac{6(p+\sigma)}{2^c}\right) \\
&\ge \frac{1}{2^{n+r\ell''_j}} \left(1 - \frac{6\ell'_j(p+\sigma)}{2^c}\right).
\end{aligned}$$

Note that, for any distinct $\tau_1, \tau_2 \in \Gamma_p^{j-1}$, $\pi \vdash \tau_1$ and $\pi \vdash \tau_2$ are mutually exclusive. Therefore, the above inequality means

$$\Pr\left[\pi \vdash \Gamma_p^j \mid \pi \vdash \Gamma_p^{j-1}\right] \ge \frac{1}{2^{n+r\ell''_j}} \left(1 - \frac{6\ell'_j(p+\sigma)}{2^c}\right). \tag{11}$$

Hence, by (10) and (11), the following holds, which means the **Claim 2** holds.

$$\Pr\left[\pi \vdash \tau_u \mid \pi \vdash (\tau_p, \tau_d)\right] \geq \frac{1}{2^{(q-q')n+r\sum_{j=q'+1}^{q}\ell_j''}}\left(1 - \frac{6\sigma(p+\sigma)}{2^c}\right).$$

with (9) and the above two claims, we have

$$\frac{\mathsf{p}_{\mathsf{re}}\left[(\tau_p, \tau_d, \tau_u)\right]}{\mathsf{p}_{\mathsf{id}}\left[(\tau_p, \tau_d, \tau_u)\right]} \geq 1 - \frac{6\sigma(p+\sigma)}{2^c}$$

Then, using the H-Coefficient technique, we can conclude the proof.

5 POSDRBG: Permutation-based DRBG with Rate 1

In this section, we propose a new permutation-based DRBG, dubbed POSDRBG. POSDRBG is a permutation-based DRBG that achieves an optimal output rate by modifying the next function of Sponge-DRBG. Notably, POSDRBG generates random numbers in a parallelizable manner, while it enjoys the same level of provable security as Sponge-DRBG. POSDRBG is defined as in Algorithm 6 (see also Figure 1).

Algorithm 6 Syntax of the POSDRBG

refresh : $\{0,1\}^n \times \{0,1\}^r \to \{0,1\}^n$
Procedure refresh$[\pi](S, I)$
1: $S \leftarrow \pi\left(S^{(r)} \oplus I \parallel S^{(c)}\right)$
2: **return** S

next : $\{0,1\}^n \times \{0,1\}^* \to \{0,1\}^n \times \{0,1\}^*$
Procedure next$[\pi](S, \mathsf{len})$
1: $\mathsf{out} \leftarrow S^{(r)}$
2: $\ell \leftarrow \lceil(\mathsf{len} - r)/n\rceil$
3: **for** $i \leftarrow 1$ to ℓ **do**
4: $\quad y \leftarrow \pi\left((S^{(r)} + i \bmod 2^r) \parallel S^{(c)}\right) \oplus 0^r \parallel S^{(c)}$
5: $\quad \mathsf{out} \leftarrow \mathsf{out} \parallel y$
6: $S \leftarrow \pi(S) \oplus 0^r \parallel S^{(c)}$
7: **return** $(S, \mathsf{out}[0 : \mathsf{len} - 1])$

For POSDRBG, we have the following lemma. The proof of (12) and (13) are deferred to the following subsection 5.1 and 5.2.

Lemma 6. *If* $p + \lceil \frac{n}{r} \rceil(\sigma_1 + \sigma_2) \leq 2^{c-1}$*, then one has*

$$\begin{aligned}
&\mathbf{Adv}^{\mathsf{POSDRBG}}_{\mathsf{M\text{-}Rec}}(p, q, \sigma_1, \sigma_2, \lambda) \\
&\leq \frac{5\lceil \frac{n}{r} \rceil (p + \lceil \frac{n}{r} \rceil (\sigma_1 + \sigma_2))(\sigma_1 + \sigma_2) + 2p(p+1)}{2^c} + \frac{(\lceil \frac{n}{r} \rceil \sigma_2)^2}{2^n} + \frac{pq}{2^\lambda}, \qquad (12) \\
&\mathbf{Adv}^{\mathsf{POSDRBG}}_{\mathsf{M\text{-}Pres}}(p, q, \sigma_1, \sigma_2) \\
&\leq \frac{(p + \lceil \frac{n}{r} \rceil (\sigma_1 + 2\sigma_2))(2q + \lceil \frac{n}{r} \rceil (7\sigma_1 + 13\sigma_2)) + 2q^2 + \lceil \frac{n}{r} \rceil \sigma_2 (\lceil \frac{n}{r} \rceil \sigma_2 + q)}{2^c} \\
&+ \frac{\lceil \frac{n}{r} \rceil \sigma_2 (\lceil \frac{n}{r} \rceil \sigma_2 + 1)}{2^n}. \qquad (13)
\end{aligned}$$

The proof of Lemma 6 follows a similar approach to the one used in the proof of Lemma 5 (see subsection 4.1 and subsection 4.2). Thus, it is recommended that the reader review the proof of Lemma 5 before proceeding. Note that, since POSDRBG can generate outputs in parallel, the proof becomes more complex than the Sponge-DRBG proof. However, the core idea remains largely the same as in the proof of Lemma 5. We will use $\sigma := \lceil \frac{n}{r} \rceil (\sigma_1 + \sigma_2)$, $\sigma'_1 := \lceil \frac{n}{r} \rceil \sigma_1$, and $\sigma'_2 := \lceil \frac{n}{r} \rceil \sigma_2$, when proving (12) and (13).

Then, by Lemma 4 and Lemma 6, we have the following theorem.

Theorem 2. *If* $p + 2\lceil \frac{n}{r} \rceil (\sigma_1 + \sigma_2) \leq 2^{c-1}$*, then one has*

$$\begin{aligned}
&\mathbf{Adv}^{\mathsf{POSDRBG}}_{rob}(p, q_1, q_2, \sigma_1, \sigma_2, \lambda) \\
&\leq \frac{4(p + 2\lceil \frac{n}{r} \rceil (\sigma_1 + \sigma_2))(p + \lceil \frac{n}{r} \rceil (7\sigma_1 + 10\sigma_2) + q_1 + 1) + 4q_1^2 + 2\lceil \frac{n}{r} \rceil \sigma_2 (q_1 + \lceil \frac{n}{r} \rceil \sigma_2)}{2^c} \\
&+ \frac{2\lceil \frac{n}{r} \rceil \sigma_2 (2\lceil \frac{n}{r} \rceil \sigma_2 + 1)}{2^n} + \frac{2(p + \lceil \frac{n}{r} \rceil (\sigma_1 + \sigma_2))q_2}{2^\lambda}.
\end{aligned}$$

5.1 Proof of (12)

Overview. Unlike Sponge-DRBG, POSDRBG supports parallel output generation using a counter. In the real world, a single query to next produces all different output blocks since the inputs to the underlying permutation are all different. So we exclude the output-colliding case by the event bad_3.

The analysis of good cases still relies on the FRESH and FREE techniques, but we introduce an additional condition, named ALIGN. This ensures that the intermediate state $\mathcal{S}$ generates all output blocks within a single query to next without conflicting with the outputs for each counter value.

Proof. Suppose that at the end of the game, the adversary $\mathcal{A}_2$ can get every entropy input that $\mathcal{A}_1$ made. For i-th M-REC call, if the output length len_i is smaller than r or $\mathsf{len}_i - r$ is not a multiple of n, the truncated part $y_i[\mathsf{len}_i :]$ is provided to the attacker. In the ideal world, it is generated randomly and provided. Then, a transcript $\tau = (\tau_p, \tau_q)$ is defined as follows.

– τ_p is a set of query results for the permutation π, (u, v) where

$$\pi(u) = v$$

holds. We denote $\tau_p = \{(u_i, v_i)\}_{i=1,\ldots,p}$.
– $\tau_q = (\mathcal{Q}_1, \cdots, \mathcal{Q}_q)$.
– $\mathcal{Q}_i$ is a tuple of query history of i-th M-REC call. The elements of $\mathcal{Q}_i$ are $(S_{i,0}, I_{i,1}, \ldots, I_{i,\ell_i-1}, S_{i,\ell_i}, y_i, \mathsf{len}_i)$ where
  - $S_{i,0}$ be a initial DRBG state of i-th M-REC that adversary choose.
  - ℓ_i be the number of queries of i-th M-REC.
  - len_i be a required output length.
  - S_{i,ℓ_i} be DRBG state that the i-th

$$(S_{i,\ell_i}, y_i) \leftarrow \text{M-REC}(S_{i,0}, I_{i,1}, \ldots, I_{i,\ell_i-1}, \mathsf{len}_i)$$

returns.
  - y_i be a value that i-th $(S_{i,\ell_i}, y_i) \leftarrow \text{M-REC}(S_{i,0}, I_{i,1}, \ldots, I_{i,\ell_i-1}, \mathsf{len}_i)$ returns. We define $t_i \leftarrow \lceil \frac{\mathsf{len}_i - r}{n} \rceil$. Then we denote $y_i \parallel y_i[\mathsf{len}_i :] = y_i^{(r)} \parallel y_i^1 \parallel y_i^2 \parallel \cdots \parallel y_i^{t_i}$ where $y_i^{(r)} \in \{0,1\}^r$ and $y_i^j \in \{0,1\}^n$ for $1 \le j \le t_i$.

For each $\mathcal{Q}_j$, define $I_{j,\ell_j} := 0^r$ and $y_j^0 := S_{j,\ell_j}$. Then define ℓ_j^c as the maximum number that satisfies $(X_{j,i-1} \oplus I_{j,i} \parallel 0^c, X_{j,i}) \in \tau_p$ for some values $X_{j,1}, \ldots, X_{j,\ell_j^c}$ with $S_{j,0} = X_{j,0}$ for all $1 \le i \le \ell_j^c \le \ell_j$.

Bad event analysis. We define bad events for M-Rec game as follows.

– bad_1: For $1 \le i \le p$, there exists $(u_i, v_i) \in \tau_p$ such that $u_i^{(c)} = v_i^{(c)}$
– bad_2: For $1 \le i \le p$, there exists $(u_i, v_i) \in \tau_p$ such that $u_i^{(c)} \in V_{i-1} \wedge v_i^{(c)} \in V_{i-1}$ where

$$V_{i-1} := \left\{ u_j^{(c)} \mid (u_j, \cdot) \in \tau_p \vee (\cdot, u_j) \in \tau_p, 1 \le j < i \right\}.$$

– bad_3: For $1 \le j \le q$, there exists $0 \le \alpha \ne \beta \le t_j$ such that $y_j^\alpha = y_j^\beta$
– bad_4^j for $1 \le j \le q : \ell_j^c = \ell_j - 1$

$$\mathsf{p}_{\mathsf{id}}[\mathsf{bad}_1] \le \frac{2p}{2^c}$$
$$\mathsf{p}_{\mathsf{id}}[\mathsf{bad}_2] \le \frac{2p^2}{2^c}$$
$$\mathsf{p}_{\mathsf{id}}[\mathsf{bad}_3] \le \sum_{j=1}^{q} \frac{(t_j+1)^2}{2^n} \le \frac{\sigma_2'^2}{2^n}$$

Define an event $\mathsf{NICE} = \neg(\mathsf{bad}_1 \vee \mathsf{bad}_2)$. Define the following graph $G = (V, E)$ based on τ_p:

– $V = \left\{ u^{(c)} \mid (u, \cdot) \in \tau_p \vee (\cdot, u) \in \tau_p \right\}$

– $E = \left\{(u^{(c)}, v^{(c)}, l) \mid (u, v) \in \tau_p \wedge l = (u^{(r)}, v^{(r)})\right\}$

Claim 1: If NICE holds, then the graph G is a forest.
Define a subgraph $G_i = (V_i, E_i)$ as

$$V_i = \left\{u_j^{(c)} \mid ((u_j, \cdot) \in \tau_p \vee (\cdot, u_j) \in \tau_p) \wedge 1 \leq j \leq i\right\}$$
$$E_i = \left\{(u_j^{(c)}, v_j^{(c)}, l_j) \mid (u_j, v_j) \in \tau_p \wedge l_j = (u_j^{(r)}, v_j^{(r)}) \wedge 1 \leq j \leq i\right\}.$$

Define an event $F_i := G_i$ is a forest. Now we use a mathematical induction. As bad_1 does not happen, self-loop does not appear in G_1. Hence F_1 holds. Assume F_{i-1} holds. To make F_i false, $(u_i, v_i) \in \tau_p$ should create a self-loop or a cycle. But bad_1 does not happen, $u_i^{(c)} \neq v_i^{(c)}$. Hence (u_i, v_i) cannot make a self-loop. To make a cycle, at least $u_i^{(c)} \in V_{i-1} \wedge v_i^{(c)} \in V_{i-1}$ is needed that is exactly bad_2. But bad_2 does not happen, therefore, F_i holds.

Claim 2: For $1 \leq j \leq q$, $\Pr\left[\mathsf{bad}_4^j \wedge \mathsf{NICE}\right] \leq \frac{p}{2^\lambda}$.

For some ℓ^j, define a potential chain as any values $v_{j,1}, \ldots, v_{j,\ell^j}$ such that $v_{j,0} := S_{j,0}$ and satisfies a below condition:

– For all $1 \leq i \leq \ell^j$, $(v_{j,i-1} \oplus \mu_{j,i} \,\|\, 0^c, v_{j,i}) \in \tau_p$

with some values $\mu_{j,1}, \ldots, \mu_{j,\ell^j}$. If NICE holds, the graph $G = (V, E)$ is a forest by the Claim 1. Hence, $S_{j,0}^{(c)} \in V$ is an element of some tree T in G. As the tree has a unique path, at most p potential chains with a starting point $v_{j,0} = S_{j,0}^{(c)}$ can exist in G.

The event bad_4^j happens if and only if $I_{j,i} = \mu_{j,i}$ for all $1 \leq i \leq \ell_j - 1$. Let $\mathcal{I}^j$ be a random variable about $I_{j,1} \,\|\, \cdots \,\|\, I_{j,\ell_j-1}$. Then, by the legitimacy condition of the adversary, the following inequality holds for any $\tau_j = (\tau_p, \tau_q \backslash Q_j)$ and any adversary's state $S_{all,j}$.

$$\mathsf{p_{id}}\left[\mathsf{bad}_4^j \mid \tau_j \wedge S_{all,j} \wedge \mathsf{NICE}\right] \leq p \cdot \mathsf{Pred}\left(\mathcal{I}^j \mid \tau_j \wedge S_{all,j} \wedge \mathsf{NICE}\right).$$

Let Γ_j be the set of all possible tuple $(\tau_j, S_{all,j})$ and $g\Gamma_j$ be the set of all possible $(\tau_j, S_{all,j})$ that satisfies NICE. Then with the above inequality, we can obtain the following inequality.

$$\begin{aligned}
\mathsf{p_{id}}\left[\mathsf{bad}_4^j \wedge \mathsf{NICE}\right] &= \sum_{(\tau_j, S_{all,j}) \in \Gamma_j} \mathsf{p_{id}}\left[\tau_j \wedge S_{all,j}\right] \cdot \mathsf{p_{id}}\left[\mathsf{bad}_4^j \wedge \mathsf{NICE} | \tau_j \wedge S_{all,j}\right] \\
&= \sum_{(\tau_j, S_{all,j}) \in g\Gamma_j} \mathsf{p_{id}}\left[\tau_j \wedge S_{all,j}\right] \cdot \mathsf{p_{id}}\left[\mathsf{bad}_4^j | \tau_j \wedge S_{all,j}\right] \\
&\leq p \sum_{(\tau_j, S_{all,j}) \in g\Gamma_j} \mathsf{p_{id}}\left[\tau_j \wedge S_{all,j}\right] \cdot \mathsf{Pred}(\mathcal{I}^j | \tau_j \wedge S_{all,j}) \\
&\leq p \cdot \mathsf{Pred}(\mathcal{I}^j | \mathcal{T}_j \wedge \mathcal{S}_{all,j}) \leq \frac{p}{2^\lambda}
\end{aligned}$$

where $\mathcal{T}_j$ is a random variable about τ_j and $\mathcal{S}_{all,j}$ is a random variable about $S_{all,j}$. Now we can bound a probability of bad events.

$$\begin{aligned}\mathsf{p}_{\mathsf{id}}[\mathsf{bad}] &= \mathsf{p}_{\mathsf{id}}[\neg\mathsf{NICE}] + \mathsf{p}_{\mathsf{id}}[\mathsf{bad}_3] + \mathsf{p}_{\mathsf{id}}\left[\mathsf{NICE} \wedge \left(\bigvee_{j=1}^{q} \mathsf{bad}_4^j\right)\right] \\ &\leq \mathsf{p}_{\mathsf{id}}[\neg\mathsf{NICE}] + \mathsf{p}_{\mathsf{id}}[\mathsf{bad}_3] + \sum_{j=1}^{q} \mathsf{p}_{\mathsf{id}}\left[\mathsf{NICE} \wedge \mathsf{bad}_4^j\right] \\ &\leq \frac{2p(p+1)}{2^c} + \frac{\sigma_2'^2}{2^n} + \frac{pq}{2^\lambda}.\end{aligned}$$

Good event analysis. For a good transcript $\tau = (\tau_p, \tau_q)$, we have

$$\begin{aligned}\frac{\mathsf{p}_{\mathsf{re}}[(\tau_p,\tau_q)]}{\mathsf{p}_{\mathsf{id}}[(\tau_p,\tau_q)]} &= \frac{\Pr[\pi \vdash \tau_p]\Pr[\pi \vdash \tau_q \mid \pi \vdash \tau_p]}{\Pr[\pi \vdash \tau_p]\Pr\left[S_{1,\ell_1},\ldots,S_{q,\ell_q},y_1,\ldots,y_q\right]} \\ &= \frac{\Pr[\pi \vdash \tau_q \mid \pi \vdash \tau_p]}{1/2^{nq+\sum_{j=1}^{q}\mathsf{len}_j}} \\ &= 2^{nq+\sum_{j=1}^{q}\mathsf{len}_j}\Pr[\pi \vdash \tau_q \mid \pi \vdash \tau_p] \qquad (14)\end{aligned}$$

For $1 \leq j \leq q$, $1 \leq w \leq j$, $\ell_w^c < i < \ell_w$ and any $s_{j,i} \in \{0,1\}^n$, let $\mathbf{S_j}$ be a tuple of $(s_{1,\ell_1^c+1},\ldots,s_{1,\ell_1-1},s_{2,\ell_2^c+1},\ldots,s_{j,\ell_j-1})$ that satisfies

$$s_{w,i} = \pi(s_{w,i-1} \oplus I_{w,i} \parallel 0^c),$$

and for $0 \leq z \leq t_w$:

$$\begin{aligned}y_w^z &= \pi((s_{w,\ell_w-1}^{(r)} + z) \parallel s_{w,\ell_w-1}^{(c)}) \oplus 0^r \parallel s_{w,\ell_w-1}^{(c)} \\ s_{w,\ell_w-1}^{(r)} &= y_w^{(r)}.\end{aligned}$$

Note that, by the definition of ℓ_w^c, $(s_{w,\ell_w^c-1} \oplus I_{w,\ell_w^c} \parallel 0^c, s_{w,\ell_w^c}, \cdot) \in \tau_p$. For each $\mathbf{S_j}$, we define a set τ_p^j as follows:

$$\begin{aligned}\tau_p^j := \tau_p \cup \{&(s_{w,i-1} \oplus I_{w,i} \parallel 0^c, s_{w,i}, 0) \mid 1 \leq w \leq j, \ell_w^c < i < \ell_w\} \\ \cup \Big\{&((s_{w,\ell_w-1}^{(r)} + z) \parallel s_{w,\ell_w-1}^{(c)}, y_w^z \oplus 0^r \parallel s_{w,\ell_w-1}^{(c)}, 0) \mid 1 \leq w \leq j, 0 \leq z \leq t_w\Big\}.\end{aligned}$$

Let Γ_p^j be a set of all possible τ_p^j that satisfies the following condition:

- For every distinct $(u,v),(u',v') \in \tau_p^j$, $u \neq u'$ and $v \neq v'$

Note that for any $\tau_p^j \in \Gamma_p^j$ and $\pi \vdash \tau_p^j$, $\pi \vdash (\tau_p, Q_1, \ldots, Q_j)$ holds. Let $\pi \vdash \Gamma_p^j$ be an event that there exists $\tau_p^j \in \Gamma_p^j$ that satisfies $\pi \vdash \tau_p^j$ by slightly abusing notation. Define $\Gamma_p^0 := \{\tau_p\}$. Then we obtain the following inequalities.

$$\begin{aligned}\Pr[\pi \vdash \tau_q \mid \pi \vdash \tau_p] &\geq \Pr\left[\pi \vdash \Gamma_p^q \mid \pi \vdash \tau_p\right] \\ &\geq \prod_{j=1}^{q} \Pr\left[\pi \vdash \Gamma_p^j \mid \pi \vdash \Gamma_p^{j-1}\right]. \qquad (15)\end{aligned}$$

For $1 \le j \le q$ and $\ell_j^c < i < \ell_j$, we recursively define a random variable $\mathcal{S}_{j,i}$ as follows.

$$\mathcal{S}_{j,i} := \pi(\mathcal{S}_{j,i-1} \oplus I_{j,i} \parallel 0^c)$$

where $\mathcal{S}_{j,\ell_j^c} = s_{j,\ell_j^c}$ and is already known to the adversary by the definition of ℓ_j^c. For any $\tau_p^{j-1} \in \Gamma_p^{j-1}$, define the following events:

- For every $\ell_j^c < i < \ell_j$, $\mathsf{FRESH}_{\mathsf{i}}^{\mathsf{j}}$: following events hold.
  - $(\mathcal{S}_{j,i-1} \oplus I_{j,i} \parallel 0^c, \cdot) \notin \tau_p^{j-1}$
  - $\mathcal{S}_{j,i-1} \oplus I_{j,i} \parallel 0^c \neq \mathcal{S}_{j,k-1} \oplus I_{j,k} \parallel 0^c$ for all $1 \le k < i$
- $\mathsf{FRESH}_{\ell_j}^{\mathsf{j}}$: following events hold.
  - $((\mathcal{S}_{j,\ell_j-1}^{(r)} + z) \parallel \mathcal{S}_{j,\ell_j-1}^{(c)}, \cdot) \notin \tau_p^{j-1}$ for all $0 \le z \le t_j$
  - $(\mathcal{S}_{j,\ell_j-1}^{(r)} + z) \parallel \mathcal{S}_{j,\ell_j-1}^{(c)} \neq \mathcal{S}_{j,k-1} \oplus I_{j,k} \parallel 0^c$ for all $1 \le k < \ell_j$ and $0 \le z \le t_j$
- $\mathsf{FREE}_{\ell_j}^{\mathsf{j}}$: following events hold.
  - $(\cdot, y_j^z \oplus 0^r \parallel \mathcal{S}_{j,\ell_j-1}^{(c)}) \notin \tau_p^{j-1}$ for all $0 \le z \le t_j$
  - $y_j^z \oplus 0^r \parallel \mathcal{S}_{j,\ell_j-1}^{(c)} \neq \mathcal{S}_{j,k}$ for all $\ell_j^c < k < \ell_j$, $0 \le z \le t_j$
  - $\mathcal{S}_{j,\ell_j-1}^{(r)} = y_j^{(r)}$
- $\mathsf{FRESH}^{\mathsf{j}} := \mathsf{FREE}_{\ell_j}^{\mathsf{j}} \wedge \left(\bigwedge_{t=\ell_j^c+1}^{\ell_j} \mathsf{FRESH}_{\mathsf{t}}^{\mathsf{j}}\right)$
- ALIGN^j : following event holds.
  - $\pi\left((\mathcal{S}_{j,\ell_j-1}^{(r)} + z) \parallel \mathcal{S}_{j,\ell_j-1}^{(c)}\right) = y_j^z \oplus 0^r \parallel \mathcal{S}_{j,\ell_j-1}^{(c)}$ for all $0 \le z \le t_j$

Then the following inequalities hold for $1 \le j \le q$ and any $\tau_p^{j-1} \in \Gamma_p^j$:

- For $\ell_j^c < i < \ell_j$,

$$\begin{aligned}
&\Pr\left[\neg\mathsf{FRESH}_{\mathsf{i}}^{\mathsf{j}} \mid \pi \vdash \tau_p^{j-1} \wedge \left(\bigwedge_{t=\ell_j^c+1}^{i-1} \mathsf{FRESH}_{\mathsf{t}}^{\mathsf{j}}\right)\right] \\
&\le \frac{p+\sigma}{2^n - (p+\sigma)} \le \frac{2(p+\sigma)}{2^n}
\end{aligned}$$

-

$$\begin{aligned}
&\Pr\left[\neg\left(\mathsf{FRESH}_{\ell_j}^{\mathsf{j}} \wedge \mathsf{FREE}_{\ell_j}^{\mathsf{j}}\right) \mid \pi \vdash \tau_{\mathsf{p}}^{\mathsf{j}-1} \wedge \left(\bigwedge_{\mathsf{t}=\ell_j^c+1}^{\ell_j-1} \mathsf{FRESH}_{\mathsf{t}}^{\mathsf{j}}\right)\right] \\
&\le \frac{2(p+\sigma)(t_j+1)}{2^n - (p+\sigma)} + \left(1 - \frac{2^c - (p+\sigma)}{2^n}\right) \\
&\le 1 - \frac{1}{2^r}\left(1 - \frac{(p+\sigma)(4t_j+5)}{2^c}\right)
\end{aligned}$$

– Considering a good transcript conditioned on $\tau \vdash \tau_p^{j-1}$ and FRESH^j, every input and output of π is different for ALIGN^j. Hence,

$$\Pr\left[\mathsf{ALIGN}^j \mid \pi \vdash \tau_p^{j-1} \wedge \mathsf{FRESH}^j\right] \geq 2^{-n(t_j+1)}$$

Using the above inequalities, for any $\tau_p^{j-1} \in \Gamma_p^{j-1}$, we can bound the following probabilities.

$$\begin{aligned}
&\Pr\left[\pi \vdash \Gamma_p^j \mid \pi \vdash \tau_p^{j-1}\right] \\
&\geq \Pr\left[\pi \vdash \Gamma_p^j \mid \pi \vdash \tau_p^{j-1} \wedge \mathsf{FRESH}^j\right] \cdot \Pr\left[\mathsf{FRESH}^j \mid \pi \vdash \tau_p^{j-1}\right] \\
&\geq \Pr\left[\mathsf{ALIGN}^j \mid \pi \vdash \tau_p^{j-1} \wedge \mathsf{FRESH}^j\right] \cdot \Pr\left[\mathsf{FRESH}^j \mid \pi \vdash \tau_p^{j-1}\right] \\
&\geq \frac{1}{2^{n+\mathsf{len}_j}} \left(1 - \frac{2(p+\sigma)}{2^n}\right)^{\ell_j - \ell_j^c - 1} \left(1 - \frac{(p+\sigma)(4t_j+5)}{2^c}\right)
\end{aligned}$$

where $\mathsf{len}_j = r + nt_j$ by its definition. Note that, for any distinct $\tau_1, \tau_2 \in \Gamma_p^{j-1}$, $\pi \vdash \tau_1$ and $\pi \vdash \tau_2$ are mutually exclusive. Therefore, the above inequality means the following.

$$\begin{aligned}
&\Pr\left[\pi \vdash \Gamma_p^j \mid \pi \vdash \Gamma_p^{j-1}\right] \\
&\geq \frac{1}{2^{n+\mathsf{len}_j}} \left(1 - \frac{2(p+\sigma)}{2^n}\right)^{\ell_j - \ell_j^c - 1} \left(1 - \frac{(p+\sigma)(4t_j+5)}{2^c}\right). \qquad (16)
\end{aligned}$$

Hence, by (15) and (16), the following holds,

$$\begin{aligned}
&\Pr\left[\pi \vdash \tau_q \mid \pi \vdash \tau_p\right] \\
&\geq \frac{1}{2^{qn+\sum_{j=1}^q \mathsf{len}_j}} \left(1 - \frac{2(p+\sigma)}{2^n}\right)^{\sum_{j=1}^q \ell_j - \ell_j^c - 1} \left(1 - \frac{5(p+\sigma)\sum_{j=1}^q (t_j+1)}{2^c}\right) \\
&\geq \frac{1}{2^{qn+\sum_{j=1}^q \mathsf{len}_j}} \left(1 - \frac{5(p+\sigma)\sigma}{2^c}\right).
\end{aligned}$$

Then, with (14) and the above inequality, we have

$$\frac{\mathsf{p}_{\mathsf{re}}\left[(\tau_p, \tau_q)\right]}{\mathsf{p}_{\mathsf{id}}\left[(\tau_p, \tau_q)\right]} \geq 1 - \frac{5(p+\sigma)\sigma}{2^c}.$$

By using the H-Coefficient technique, we can conclude the proof.

5.2 Proof of (13)

Overview. Similarly, a key distinction lies in handling outputs generated within a single query to next. We define bad cases to ensure that the outputs from different next queries remain distinct and do not overlap with states known to the adversary.

Since the initial state $S_{i,0}$ is provided after the adversary finishes making their queries, querying ROR immediately after INTER allows the adversary to know the feed-forwarding value $S_{i,0}^{(c)}$. By XORing the outputs from ROR with $0^r \parallel S_{i,0}^{(c)}$, the adversary obtains the corresponding permutation outputs. Therefore, we define a bad case where these outputs overlap with the results from primitive queries. The ALIGN condition is also applied to ensure output consistency.

Proof. Suppose that at the end of the game, the adversary can get the following values.

- DRBG state immediately after INIT or INTER queries. Note that if an adversary makes $\mathrm{ROR}_{\mathrm{mpres}}$ exactly after the INIT or INTER query, then only the lower c-bit of the state is given to the adversary. This is because an adversary can obtain the upper r-bit of the state by the $\mathrm{ROR}_{\mathrm{mpres}}$. Hence, an adversary can fully recover the $n = r + c$-bit state.
- DRBG state immediately after the last $\mathrm{ROR}_{\mathrm{mpres}}$ query before each INTER query and all states between that $\mathrm{ROR}_{\mathrm{mpres}}$ and the INTER(= all query results for π starting from the DRBG state after the last $\mathrm{ROR}_{\mathrm{mpres}}$ query). Note that if an adversary does not make $\mathrm{ROR}_{\mathrm{mpres}}$ query, then we give every state between two INTER queries.
- For each $\mathrm{ROR}_{\mathrm{mpres}}$ call, if the output length len is smaller than r or $\mathsf{len} - r$ is not a multiple of n, the truncated part is provided to the attacker. In the ideal world, it is generated randomly and provided. Hence, we assume every $\mathrm{ROR}_{\mathrm{mpres}}$ output length len is exactly r or $\mathsf{len} - r$ is a multiple of n.

Then the transcript can be constructed as

$$\tau = (\tau_p, \tau_q),$$

where,

- τ_p is a set of query results for the permutation π, (u, v) where

$$\pi(u) = v$$

holds. We denote $\tau_p = \{(u_i, v_i)\}_{i=1,\ldots,p'}$, where $p' - p$ is the number of additionally given permutation results at the end of the game. Note that $p' \leq p + \sigma_1'$.
- $\tau_q = (\mathcal{Q}_1, \cdots, \mathcal{Q}_q)$.
- $\mathcal{Q}_i$ is a tuple of query history between $i-1$-th INTER and i-th INTER (INIT can be counted as 0-th INTER). The elements of $\mathcal{Q}_i$ are

$$(S_{i,0}, Z_{i,1}, \cdots, Z_{i,\ell_i'}, S_{i,\ell_i'}, b_{i,1}, \cdots, b_{i,\ell_i'})$$

where
  - $S_{i,0}$ be DRBG state immediately after $i-1$-th INTER queries.
  - ℓ_i' be the number of $\mathrm{REF}_{\mathrm{mpres}}$ or $\mathrm{ROR}_{\mathrm{mpres}}$ queries between $i-1$-th INTER and the last $\mathrm{ROR}_{\mathrm{mpres}}$ query before i-th INTER query.

- S_{i,ℓ'_i} be DRBG state immediately after the last $\mathrm{ROR}_{\mathrm{mpres}}$ query before i-th INTER query.
- $b_{i,j} \in \{0,1\}$ be the indicator for the j-th $\mathrm{REF}_{\mathrm{mpres}}$ or $\mathrm{ROR}_{\mathrm{mpres}}$ query after $i-1$-th INTER. If $b_{i,j} = 1$, $\mathrm{REF}_{\mathrm{mpres}}$ is queried. If $b_{i,j} = 0$, $\mathrm{ROR}_{\mathrm{mpres}}$ is queried.
- $Z_{i,j}$ be the input or output value for the j-th $\mathrm{REF}_{\mathrm{mpres}}$ or $\mathrm{ROR}_{\mathrm{mpres}}$ query after $i-1$-th INTER. If $b_{i,j} = 1$, $Z_{i,j}$ be the $\mathrm{REF}_{\mathrm{mpres}}$ input, if $b_{i,j} = 0$, $Z_{i,j}$ be the $\mathrm{ROR}_{\mathrm{mpres}}$ output.

We define $Z'_{i,j}$ as $Z^{(r)}_{i,j} \parallel Z'_{i,j} \leftarrow Z_{i,j}$. Also, we define $t_{i,j}$ as a block length of $Z'_{i,j}$. If $b_{i,j} = 1$, $t_{i,j} := 0$. Then, if $b_{i,j} = 0$, we can separate n-bit output blocks of ROR query as $Y^1_{i,j} \parallel Y^2_{i,j} \parallel \cdots \parallel Y^{t_{i,j}}_{i,j} \xleftarrow{n} Z'_{i,j}$. Also $Y^0_{i,j} \leftarrow S_{i,j}$. Let ℓ''_i be the number of $b_{i,j}$ with $b_{i,j} = 0$ for $2 \le j \le \ell'_i$.

Without loss of generality, suppose for some q' with $1 \le q' \le q$, $\mathcal{Q}_i$ with $1 \le i \le q'$ has $\ell'_i = 1$ and $b_{i,1} = 0$, $\mathcal{Q}_j$ with $q'+1 \le j \le q$ doesn't. Then we can rewrite the transcript as

$$\tau = (\tau_p, \tau_d, \tau_u),$$

where $\tau_d = (\mathcal{Q}_1, \cdots, \mathcal{Q}_{q'})$ and $\tau_u = (\mathcal{Q}_{q'+1}, \cdots, \mathcal{Q}_q)$.

Bad event analysis. We define bad events for M-Pres game as follows.

- bad_1 : For any $1 \le i \le q$, there exists $(\cdot \parallel u, \cdot) \in \tau_p$ such that $S^{(c)}_{i,0} = u$.
- bad'_1 : For any $1 \le i \ne j \le q$, $S^{(c)}_{i,0} = S^{(c)}_{j,0}$ holds.
- bad_2 : For any $1 \le i \le q$ with $b_{i,1} = 0$, there exists $1 \le j \le t_{i,1}$ and $(\cdot, v) \in \tau_p$ such that $Y^j_{i,1} \oplus 0^r \parallel S^{(c)}_{i,0} = v$.
- bad'_2 : For any $1 \le i, j \le q$ with $b_{i,1} = b_{j,1} = 0$, there exists $1 \le \alpha \le t_{i,1}$ and $1 \le \beta \le t_{j,1}$ such that $Y^\alpha_{i,1} \oplus 0^r \parallel S^{(c)}_{i,0} = Y^\beta_{j,1} \oplus 0^r \parallel S^{(c)}_{j,0}$ and $(i,\alpha) \ne (j,\beta)$.
- bad_3 : For any $1 \le i \le q'$, there exists $(\cdot, v) \in \tau_p$ such that $S_{i,1} \oplus 0^r \parallel S^{(c)}_{i,0} = v$.
- bad'_3 : For any $1 \le i \ne j \le q'$, $S_{i,1} \oplus 0^r \parallel S^{(c)}_{i,0} = S_{j,1} \oplus 0^r \parallel S^{(c)}_{j,0}$.
- bad_4 : For any $1 \le i \le q'$ and $q'+1 \le j \le q$, there exists $1 \le z \le t_{j,1}$ such that $S_{i,1} \oplus 0^r \parallel S^{(c)}_{i,0} = Y^z_{j,1} \oplus 0^r \parallel S^{(c)}_{j,0}$
- bad_5 : For any $q'+1 \le i \le q$, there exists $1 \le j \le t_{i,\ell'_i}$ such that $S_{i,\ell'_i} = Y^j_{i,\ell'_i}$.
- bad_6 : For any $q'+1 \le i \le q$ and $1 \le j \le \ell'_i$, there exists $1 \le k \ne w \le t_{i,j}$ such that $Y^k_{i,j} = Y^w_{i,j}$.

$$\mathsf{p}_{\mathsf{id}}[\mathsf{bad}_1] \le \frac{(p+\sigma'_1)q}{2^c}$$

$$\mathsf{p}_{\mathsf{id}}[\mathsf{bad}'_1] \le \frac{q^2}{2^c}$$

$$\mathsf{p}_{\mathsf{id}}[\mathsf{bad}_2] \le \sum_{i=1}^{q} \frac{(p+\sigma'_1)t_{i,1}}{2^c} \le \frac{(p+\sigma'_1)\sigma'_2}{2^c}$$

$$\mathsf{p}_{\mathsf{id}}\left[\mathsf{bad}_2'\right] \leq \sum_{i=1}^{q}\sum_{j=1}^{q}\frac{t_{i,1}t_{j,1}}{2^c} \leq \frac{\sigma_2'^2}{2^c}$$

$$\mathsf{p}_{\mathsf{id}}\left[\mathsf{bad}_3\right] \leq \sum_{i=1}^{q'}\frac{(p+\sigma_1')}{2^c} \leq \frac{(p+\sigma_1')q'}{2^c}$$

$$\mathsf{p}_{\mathsf{id}}\left[\mathsf{bad}_3'\right] \leq \frac{q'^2}{2^c}$$

$$\mathsf{p}_{\mathsf{id}}\left[\mathsf{bad}_4\right] \leq \sum_{i=1}^{q'}\sum_{j=q'+1}^{q}\frac{t_{j,1}}{2^c} \leq \frac{q'\sigma_2'}{2^c}$$

$$\mathsf{p}_{\mathsf{id}}\left[\mathsf{bad}_5\right] \leq \sum_{i=1}^{q}\frac{t_{i,1}}{2^n} \leq \frac{\sigma_2'}{2^n}$$

$$\mathsf{p}_{\mathsf{id}}\left[\mathsf{bad}_6\right] \leq \sum_{i=1}^{q}\sum_{j=1}^{\ell_i'}\frac{t_{i,j}^2}{2^n} \leq \frac{\sigma_2'^2}{2^n}$$

GOOD EVENT ANALYSIS. For a good transcript $\tau = (\tau_p, \tau_d, \tau_u)$, we have

$$\begin{aligned}
&\frac{\mathsf{p}_{\mathsf{re}}\left[(\tau_p,\tau_d,\tau_u)\right]}{\mathsf{p}_{\mathsf{id}}\left[(\tau_p,\tau_d,\tau_u)\right]} \qquad (17)\\
&= \frac{\Pr\left[\pi \vdash \tau_p\right]\Pr\left[\bigwedge_{j=1}^{q} S_{j,0}\right]\Pr\left[\pi \vdash \tau_d \mid \pi \vdash \tau_p\right]\Pr\left[\pi \vdash \tau_u \mid \pi \vdash (\tau_p,\tau_d)\right]}{\Pr\left[\pi \vdash \tau_p\right]\Pr\left[\bigwedge_{j=1}^{q} S_{j,0}\right]\Pr\left[\tau_d\right]\prod_{i=q'+1}^{q}\Pr\left[\mathcal{Q}_i\right]}\\
&= \frac{\Pr\left[\pi \vdash \tau_d \mid \pi \vdash \tau_p\right]}{1/2^{nq'+\sum_{i=1}^{q'} t_{i,1}}} \cdot \frac{\Pr\left[\pi \vdash \tau_u \mid \pi \vdash (\tau_p,\tau_d)\right]}{\prod_{i=q'+1}^{q} 1/2^n \cdot 1/2^{\ell_i'' r} \cdot 1/2^{n\sum_{j=1}^{\ell_i'} t_{i,j}}}\\
&= 2^{n(q+\sum_{i=1}^{q}\sum_{j=1}^{\ell_i'} t_{i,j})+r\sum_{i=q'+1}^{q}\ell_i''}\\
&\times \Pr\left[\pi \vdash \tau_d \mid \pi \vdash \tau_p\right]\Pr\left[\pi \vdash \tau_u \mid \pi \vdash (\tau_p,\tau_d)\right]. \qquad (18)
\end{aligned}$$

For $1 \leq j \leq q'$, define a set $\mathcal{Y}_{j,1}$ and $\mathcal{Y}_{j,1}^z$ as follows:

$$\begin{aligned}
\mathcal{Y}_{j,1} &:= \left\{\left((S_{j,0}^{(r)}+k) \parallel S_{j,0}^{(c)}, Y_{j,1}^k \oplus 0^r \parallel S_{j,0}^{(c)}\right) \mid 0 \leq k \leq t_{j,1}\right\},\\
\mathcal{Y}_{j,1}^z &:= \left\{\left((S_{j,0}^{(r)}+k) \parallel S_{j,0}^{(c)}, Y_{j,1}^k \oplus 0^r \parallel S_{j,0}^{(c)}\right) \mid 0 \leq k \leq z\right\}.
\end{aligned}$$

Claim 1 : $\Pr\left[\pi \vdash \tau_d \mid \pi \vdash \tau_p\right] \geq 2^{-n\left(q'+\sum_{j=1}^{q'} t_{j,1}\right)}$.
Because τ is a good transcript, for $1 \leq j \leq q'$ and $0 \leq z \leq t_{j,1}$,

$$\Pr\left[\pi((S_{j,0}^{(r)}+z) \parallel S_{j,0}^{(c)}) = Y_{j,1}^z \oplus 0^r \parallel S_{j,0}^{(c)} \mid \pi \vdash (\tau_p, \mathcal{Q}_1, \cdots, \mathcal{Q}_{j-1}, \mathcal{Y}_{j,1}^{z-1})\right] \geq \frac{1}{2^n}.$$

Claim 2 :

$$\Pr\left[\pi \vdash \tau_u \mid \pi \vdash (\tau_p, \tau_d)\right] \geq \frac{1}{2^{(q-q')n + r\sum_{i=q'+1}^{q} \ell_i'' + n\sum_{j=q'+1}^{q}\sum_{i=1}^{\ell_j'} t_{j,i}}} \times \left(1 - \frac{5p\sigma_1' + 5\sigma_1'^2 + 23\sigma_1'\sigma_2' + 9p\sigma_2' + 22\sigma_2'^2}{2^c}\right).$$

Let $\tau_p^{q'} := \tau_p \cup \left(\bigcup_{j=1}^{q'} \mathcal{Y}_{j,1}\right)$ then it is trivial that $\pi \vdash \tau_p^{q'} = \pi \vdash (\tau_p, \tau_d)$.

For $q'+1 \leq j \leq q$, $q'+1 \leq w \leq j$, $1 \leq i \leq \ell_w'$, and any $S_{j,i} \in \{0,1\}^n$, let $\mathbf{S_j}$ be a tuple of $(S_{q'+1,1}, S_{q'+1,2}, \ldots, S_{q'+1,\ell_{q'+1}'-1}, S_{q'+2,1}, \ldots, S_{j,\ell_j'-1})$ that satisfies the followings if $b_{w,i} = 1$:

$$\pi(S_{w,i-1} \oplus Z_{w,i} \parallel 0^c) = S_{w,i}$$

also if $b_{w,i} = 0$, for $0 \leq z \leq t_{w,i}$:

$$S_{w,i-1}^{(r)} = Z_{w,i}^{(r)},$$
$$\pi\left((S_{w,i-1}^{(r)} + z) \parallel S_{w,i-1}^{(c)}\right) = Y_{w,i}^{z} \oplus 0^r \parallel S_{w,i-1}^{(c)}.$$

For each $\mathbf{S}_j$, we define a set τ_p^j as follows:

$$\begin{aligned}\tau_p^j &:= \tau_p^{q'} \\ &\cup \left\{ \bigcup_{w=q'+1}^{j} \bigcup_{i=1}^{\ell_w'} \left(S_{w,i-1} \oplus b_{w,i}^r \& Z_{w,i}^{(r)} \parallel 0^c, S_{w,i} \oplus 0^r \parallel \left((1^c \oplus b_{w,i}^c) \& S_{w,i-1}^{(c)}\right)\right) \right\} \\ &\cup \left\{ \bigcup_{w=q'+1}^{j} \bigcup_{\substack{1 \leq i \leq \ell_w', \\ b_{w,i}=0}} \bigcup_{z=1}^{t_{w,i}} \left((S_{w,i-1}^{(r)} + z) \parallel S_{w,i-1}^{(c)}, Y_{w,i}^{z} \oplus 0^r \parallel S_{w,i-1}^{(c)}\right) \right\}.\end{aligned}$$

Let Γ_p^j be a set of all possible τ_p^j that satisfies the following condition:

- For every distinct $(u,v), (u',v') \in \tau_p^j$, $u \neq u'$ and $v \neq v'$.
- For every $(u, \cdot) \in \tau_p^j$, $S_{i,0} \oplus Z_{i,1} \parallel 0^c \neq u$ for $j < i \leq q$ with $b_{i,1} = 1$.
- For every $(u, \cdot) \in \tau_p^j$, $(S_{i,0} + z) \parallel S_{i,0}^{(c)} \neq u$ for $j < i \leq q$ and $0 \leq z \leq t_{i,1}$ with $b_{i,1} = 0$.

Note that for any $\tau_p^j \in \Gamma_p^j$ and $\pi \vdash \tau_p^j$, $\pi \vdash (\tau_p, \tau_d, Q_{q'+1}, \ldots, Q_j)$ holds. Let $\pi \vdash \Gamma_p^j$ be an event that there exists $\tau_p^j \in \Gamma_p^j$ which satisfies $\pi \vdash \tau_p^j$ by slightly abusing notation. Define $\Gamma_p^{q'} = \{\tau_p^{q'}\}$. Then we obtain the following inequalities.

$$\begin{aligned}\Pr\left[\pi \vdash \tau_u \mid \pi \vdash (\tau_p, \tau_d)\right] &\geq \Pr\left[\pi \vdash \tau_u \mid \pi \vdash \tau_p^{q'}\right] \\ &\geq \Pr\left[\pi \vdash \Gamma_p^q \mid \pi \vdash \tau_p^{q'}\right] \\ &\geq \prod_{j=q'+1}^{q} \Pr\left[\pi \vdash \Gamma_p^j \mid \pi \vdash \Gamma_p^{j-1}\right]. \qquad (19)\end{aligned}$$

For $q'+1 \le j \le q$ and $1 \le i < \ell'_j$, we recursively define a random variable $\mathcal{S}_{j,i}$ as

$$\mathcal{S}_{j,i} := \pi(\mathcal{S}_{j,i-1} \oplus b^r_{j,i} \& Z^{(r)}_{j,i} \parallel 0^c) \oplus 0^r \parallel \left((1^c \oplus b^c_{j,i}) \& \mathcal{S}^{(c)}_{j,i-1}\right)$$

where $\mathcal{S}_{j,0} = S_{j,0}$.

For any $\tau_p^{j-1} \in \Gamma_p^{j-1}$, define the following events:

- FRESH^j_i : following events hold.
  - $\left((\mathcal{S}^{(r)}_{j,i-1} + z) \parallel \mathcal{S}^{(c)}_{j,i-1} \oplus b^r_{j,i} \& Z^{(r)}_{j,i} \parallel 0^c, \cdot\right) \notin \tau_p^{j-1}$ for all $0 \le z \le t_{j,i}$,
  - $(\mathcal{S}^{(r)}_{j,i-1} + z_i) \parallel \mathcal{S}^{(c)}_{j,i-1} \oplus b^r_{j,i} \& Z^{(r)}_{j,i} \parallel 0^c \ne (\mathcal{S}^{(r)}_{j,k-1} + z_k) \parallel \mathcal{S}^{(c)}_{j,k-1} \oplus b^r_{j,k} \& Z^{(r)}_{j,k} \parallel 0^c$ for all $1 \le k < i$, $0 \le z_i \le t_{j,i}$, and $0 \le z_k \le t_{j,k}$,
  - If $b_{j,i} = 0$, $\mathcal{S}^{(r)}_{j,i-1} = Z^{(r)}_{j,i}$.
- FREE^j_i: following events hold.
  - If $b_{j,i} = 0$, $(\cdot, Y^z_{j,i} \oplus 0^r \parallel \mathcal{S}^{(c)}_{j,i-1}) \notin \tau_p^{j-1}$ for all $1 \le z \le t_{j,i}$,
  - If $b_{j,i} = 0$, $Y^\alpha_{j,i} \oplus 0^r \parallel \mathcal{S}^{(c)}_{j,i-1} \ne Y^\beta_{j,k} \oplus 0^r \parallel \mathcal{S}^{(c)}_{j,k-1}$ for all $1 \le k < i$, $1 \le \alpha \le t_{j,i}$, and $1 \le \beta \le t_{j,k}$.
  - If $b_{j,i-1} = 0$ and $2 \le i < \ell'_j$, $\mathcal{S}_{j,i-1} \ne Y^z_{j,i-1}$ for all $1 \le z \le t_{j,i-1}$.
- $\mathsf{FREE}^j_{\ell'_j}$: following events hold.
  - $(\cdot, Y^z_{j,\ell'_j} \oplus 0^r \parallel \mathcal{S}^{(c)}_{j,\ell'_j - 1}) \notin \tau_p^{j-1}$ for all $1 \le z \le t_{j,\ell'_j}$,
  - $Y^\alpha_{j,\ell'_j} \oplus 0^r \parallel \mathcal{S}^{(c)}_{j,\ell'_j - 1} \ne Y^\beta_{j,k} \oplus 0^r \parallel \mathcal{S}^{(c)}_{j,k-1}$ for all $1 \le k < \ell'_j$, $1 \le \alpha \le t_{j,\ell'_j}$, and $1 \le \beta \le t_{j,k}$.
  - $(\cdot, S_{j,\ell'_j} \oplus 0^r \parallel \mathcal{S}^{(c)}_{j,\ell'_j - 1}) \notin \tau_p^{j-1}$
  - $S_{j,\ell'_j} \oplus 0^r \parallel \mathcal{S}^{(c)}_{j,\ell'_j - 1} \ne Y^z_{j,k} \oplus 0^r \parallel \mathcal{S}^{(c)}_{j,k-1}$ for all $1 \le k < \ell'_j$, and $1 \le z \le t_{j,k}$
  - $S_{j,\ell'_j} \oplus 0^r \parallel \mathcal{S}^{(c)}_{j,\ell'_j - 1} \ne \mathcal{S}_{j,k} \oplus 0^r \parallel \left((1^c \oplus b^c_{j,k}) \& \mathcal{S}^{(c)}_{j,k-1}\right)$ for all $1 \le k \le \ell'_j - 1$
  - If $b_{j,\ell'_j - 1} = 0$, $\mathcal{S}_{j,\ell'_j - 1} \ne Y^z_{j,\ell'_j - 1}$ for all $1 \le z \le t_{j,\ell'_j - 1}$.
- $\mathsf{FRESH}^j = \bigwedge_{k=1}^{\ell'_j} \left(\mathsf{FRESH}^j_k \wedge \mathsf{FREE}^j_k\right)$.
- ALIGN^j: following events hold.
  - $\pi\left((\mathcal{S}^{(r)}_{j,i-1} + z) \parallel \mathcal{S}^{(c)}_{j,i-1}\right) = Y^z_{j,i} \oplus 0^r \parallel \mathcal{S}^{(c)}_{j,i-1}$ for all $1 \le i \le \ell'_j$ and $1 \le z \le t_{j,i}$,
  - $\pi(\mathcal{S}_{j,\ell'_j - 1}) = S_{j,\ell'_j} \oplus 0^r \parallel \mathcal{S}^{(c)}_{j,\ell'_j - 1}$.

Note that, as bad events does not happen, $\Pr\left[\mathsf{FRESH}^j_1 \mid \pi \vdash \tau_p^{j-1}\right] = 1$ and $\Pr\left[\mathsf{FREE}^j_1 \mid \pi \vdash \tau_p^{j-1}\right] = 1$ for every $q'+1 \le j \le q$.

– For $2 \le i \le \ell'_j - 1$,

$$\begin{aligned}
&\Pr\left[\neg\mathsf{FRESH}^{\mathsf{j}}_{\mathsf{i}} \mid \pi \vdash \tau_p^{j-1} \wedge b_{j,i} = 0 \wedge \left(\bigwedge_{t=1}^{i-1} \mathsf{FRESH}^{\mathsf{j}}_{\mathsf{t}}\right) \wedge \left(\bigwedge_{t=1}^{i-1} \mathsf{FREE}^{j}_{t}\right)\right] \\
&\le \frac{(p+\sigma'_1+2\sigma'_2)(t_{j,i}+1)}{2^r(2^c-(p+\sigma'_1+\sigma'_2))} + \left(1 - \frac{2^c-(p+\sigma'_1+\sigma'_2)}{2^n}\right) \\
&\le 1 - \frac{1}{2^r}\left(1 - \frac{(p+\sigma'_1+2\sigma'_2)(2t_{j,i}+3)}{2^c}\right), \\
&\Pr\left[\neg\mathsf{FRESH}^{\mathsf{j}}_{\mathsf{i}} \mid \pi \vdash \tau_p^{j-1} \wedge b_{j,i} = 1 \wedge \left(\bigwedge_{t=1}^{i-1} \mathsf{FRESH}^{\mathsf{j}}_{\mathsf{t}}\right) \wedge \left(\bigwedge_{t=1}^{i-1} \mathsf{FREE}^{j}_{t}\right)\right] \\
&\le \frac{p+\sigma'_1+2\sigma'_2}{2^r(2^c-(p+\sigma'_1+\sigma'_2))} \le \frac{2(p+\sigma'_1+2\sigma'_2)}{2^n}, \\
&\Pr\left[\neg\mathsf{FREE}^{\mathsf{j}}_{\mathsf{i}} \mid \pi \vdash \tau_p^{j-1} \wedge \left(\bigwedge_{t=1}^{i-1} \mathsf{FRESH}^{\mathsf{j}}_{\mathsf{t}}\right) \wedge \left(\bigwedge_{t=1}^{i-1} \mathsf{FREE}^{j}_{t}\right)\right] \\
&\le \frac{(p+\sigma'_1+2\sigma'_2)t_{j,i}}{2^r(2^c-(p+\sigma'_1+\sigma'_2))} + \frac{t_{j,i-1}}{2^n-(p+\sigma'_1+\sigma'_2)} \\
&\le \frac{2(p+\sigma'_1+2\sigma'_2)t_{j,i}+2\sigma'_2}{2^n}.
\end{aligned}$$

Note that if $b_{j,i} = 1$, $t_{j,i} = 0$. Therefore, we conclude that

$$\begin{aligned}
&\Pr\left[\mathsf{FRESH}^{\mathsf{j}}_{\mathsf{i}} \wedge \mathsf{FREE}^{\mathsf{j}}_{\mathsf{i}} \mid \pi \vdash \tau_p^{j-1} \wedge b_{j,i} = 0 \wedge \left(\bigwedge_{t=1}^{i-1} \mathsf{FRESH}^{\mathsf{j}}_{\mathsf{t}} \wedge \mathsf{FREE}^{j}_{t}\right)\right] \\
&\ge \frac{1}{2^r}\left(1 - \frac{(p+\sigma'_1+2\sigma'_2)(4t_{j,i}+3)+2\sigma'_2}{2^c}\right), \\
&\Pr\left[\mathsf{FRESH}^{\mathsf{j}}_{\mathsf{i}} \wedge \mathsf{FREE}^{\mathsf{j}}_{\mathsf{i}} \mid \pi \vdash \tau_p^{j-1} \wedge b_{j,i} = 1 \wedge \left(\bigwedge_{t=1}^{i-1} \mathsf{FRESH}^{\mathsf{j}}_{\mathsf{t}} \wedge \mathsf{FREE}^{j}_{t}\right)\right] \\
&\ge 1 - \frac{2(p+\sigma'_1+2\sigma'_2)+2\sigma'_2}{2^n}.
\end{aligned}$$

–

$$\begin{aligned}
&\Pr\left[\mathsf{FRESH}^{\mathsf{j}}_{\ell'_j} \wedge \mathsf{FREE}^{j}_{\ell'_j} \mid \pi \vdash \tau_p^{j-1} \wedge b_{j,\ell'_j} = 0 \wedge \left(\bigwedge_{t=1}^{\ell'_j-1} \mathsf{FRESH}^{\mathsf{j}}_{\mathsf{t}} \wedge \mathsf{FREE}^{\mathsf{j}}_{\mathsf{t}}\right)\right] \\
&\ge \frac{1}{2^r}\left(1 - \frac{(p+\sigma'_1+2\sigma'_2)(4t_{j,\ell'_j}+5)+4\sigma'_2}{2^c}\right).
\end{aligned}$$

– Considering a good transcript conditioned on $\pi \vdash \tau_p^{j-1}$ and FRESH^j, every input and output of π is different for ALIGN^j. Hence,

$$\Pr\left[\mathsf{ALIGN}^j \mid \pi \vdash \tau_p^{j-1} \wedge \mathsf{FRESH}^j\right] \geq 2^{-n(1+\sum_{i=1}^{\ell'_j} t_{j,i})}.$$

Using the above inequalities, for any $\tau_p^{j-1} \in \Gamma_p^{j-1}$, we can bound the below probabilities.

$$\begin{aligned}
&\Pr\left[\pi \vdash \Gamma_p^j \mid \pi \vdash \tau_p^{j-1}\right] \\
&\geq \Pr\left[\pi \vdash \Gamma_p^j \mid \pi \vdash \tau_p^{j-1} \wedge \mathsf{FRESH}^j\right] \cdot \Pr\left[\mathsf{FRESH}^j \mid \pi \vdash \tau_p^{j-1}\right] \\
&\geq \Pr\left[\mathsf{ALIGN}^j \mid \pi \vdash \tau_p^{j-1} \wedge \mathsf{FRESH}^j\right] \cdot \Pr\left[\mathsf{FRESH}^j \mid \pi \vdash \tau_p^{j-1}\right] \\
&\geq \frac{1}{2^{n(1+\sum_{i=1}^{\ell'_j} t_{j,i})}} \Pr\left[\mathsf{FRESH}^j \mid \pi \vdash \tau_p^{j-1}\right] \\
&\geq \frac{1}{2^{r\ell''_j + n(1+\sum_{i=1}^{\ell'_j} t_{j,i})}} \left(\prod_{\substack{2 \leq i < \ell'_j \\ b_{j,i}=0}} \left(1 - \frac{(p+\sigma'_1+2\sigma'_2)(4t_{j,i}+3)+2\sigma'_2}{2^c}\right)\right) \\
&\cdot \left(\prod_{\substack{2 \leq i < \ell'_j \\ b_{j,i}=1}} \left(1 - \frac{2(p+\sigma'_1+2\sigma'_2)+2\sigma'_2}{2^n}\right)\right) \\
&\cdot \left(1 - \frac{(p+\sigma'_1+2\sigma'_2)(4t_{j,\ell'_j}+5)+4\sigma'_2}{2^c}\right) \\
&\geq \frac{1}{2^{n(1+\sum_{i=1}^{\ell'_j} t_{j,i})}} \frac{1}{2^{\ell''_j r}} \left(1 - \frac{2(p+\sigma'_1+3\sigma'_2)(\ell'_j - \ell''_j - 1)}{2^n}\right) \\
&\cdot \left(1 - \frac{(5p+5\sigma'_1+14\sigma'_2)\ell''_j + 4(p+\sigma'_1+2\sigma'_2)\sum_{i=2}^{\ell'_j} t_{j,i}}{2^c}\right) \\
&\geq \frac{1}{2^{n(1+\sum_{i=1}^{\ell'_j} t_{j,i})}} \frac{1}{2^{\ell''_j r}} \left(1 - \frac{(5p+5\sigma'_1+14\sigma'_2)\ell'_j + 4(p+\sigma'_1+2\sigma'_2)\sum_{i=2}^{\ell'_j} t_{j,i}}{2^c}\right).
\end{aligned}$$

Note that, for any distinct $\tau_1, \tau_2 \in \Gamma_p^{j-1}$, $\pi \vdash \tau_1$ and $\pi \vdash \tau_2$ are mutually exclusive. Therefore, the above inequality means

$$\begin{aligned}
\Pr\left[\pi \vdash \Gamma_p^j \mid \pi \vdash \Gamma_p^{j-1}\right] \geq{}& \frac{1}{2^{n(1+\sum_{i=1}^{\ell'_j} t_{j,i}) + \ell''_j r}} \\
&\cdot \left(1 - \frac{(5p+5\sigma'_1+14\sigma'_2)\ell'_j + 4(p+\sigma'_1+2\sigma'_2)\sum_{i=2}^{\ell'_j} t_{j,i}}{2^c}\right). \quad (20)
\end{aligned}$$

Hence, by (19) and (20), the following holds, which means the **Claim 2** holds.

$$\Pr\left[\pi \vdash \tau_u \mid \pi \vdash (\tau_p, \tau_d)\right] \geq \frac{1}{2^{(q-q')n+r\sum_{i=q'+1}^{q}\ell_i''+\sum_{j=q'+1}^{q}\sum_{i=2}^{\ell_j'}t_{j,i}}} \cdot \left(1-\frac{5p\sigma_1'+5\sigma_1'^2+23\sigma_1'\sigma_2'+9p\sigma_2'+22\sigma_2'^2}{2^c}\right).$$

Combined with (18) and the above two claims, we have

$$\frac{\mathsf{p}_{\mathsf{re}}\left[(\tau_p,\tau_d,\tau_u)\right]}{\mathsf{p}_{\mathsf{id}}\left[(\tau_p,\tau_d,\tau_u)\right]} \geq 1-\frac{5p\sigma_1'+5\sigma_1'^2+23\sigma_1'\sigma_2'+9p\sigma_2'+22\sigma_2'^2}{2^c},$$

and by the H-Coefficient technique, we can conclude the proof.

6 Attacks on Sponge-DRBG and POSDRBG

We present two attacks that apply to both Sponge-DRBG and POSDRBG. In particular, Attack 1 is based on the strategy from the CBC-Extractor attack [8]. In the CBC-Extractor attack, the adversary has full control over the input to the underlying primitive, allowing it to efficiently create an arbitrary input sequence through a chain of primitive queries, leading to an attack complexity of $O(1)$. In contrast, in Sponge-DRBG and POSDRBG, the adversary is able to control only the upper r bits of the input, making a direct application of the CBC-Extractor attack infeasible.

To overcome this restriction, our attack generates a collision on the lower c bits using $O(2^{\frac{c}{2}})$ primitive queries. In this way, the complexity of Attack 1 is $O(\lambda \cdot 2^{\frac{c}{2}})$, and for practical values of λ (e.g., 256 bits), the complexity becomes $O(2^{\frac{c}{2}})$.

Attack 1: A λ-legitimate adversary $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$ wins the robustness game with high probability by performing the following steps.

1. $\mathcal{A}_1$ defines $S_{1,1} \leftarrow 0^n$.
2. Using primitive queries, $\mathcal{A}_1$ finds two entropy inputs $I_{1,1} \neq I_{1,2}$ such that:
$$\pi(S_{1,1} \oplus I_{1,1} \parallel 0^c)^{(c)} = \pi(S_{1,1} \oplus I_{1,2} \parallel 0^c)^{(c)},$$
ensuring a collision in the lower c bits. Then $\mathcal{A}_1$ sets
$$S_{2,1} \leftarrow \pi(S_{1,1} \oplus I_{1,1} \parallel 0^c),$$
$$S_{2,2} \leftarrow \pi(S_{1,1} \oplus I_{1,2} \parallel 0^c).$$
Notably, $S_{2,1}^{(c)} = S_{2,2}^{(c)}$, and this step requires $O(2^{c/2})$ primitive queries.
3. For $i \leftarrow 2$ to λ, $\mathcal{A}_1$ performs the following steps:
   (a) Using primitive queries, $\mathcal{A}_1$ finds two entropy inputs $I_{i,1} \neq I_{i,2}$ such that:
   $$\pi(S_{i,1} \oplus I_{i,1} \parallel 0^c)^{(c)} = \pi(S_{i,1} \oplus I_{i,2} \parallel 0^c)^{(c)}.$$
   This step requires $O(2^{c/2})$ primitive queries.

(b) $\mathcal{A}_1$ sets

$$S_{i+1,1} \leftarrow \pi(S_{i,1} \oplus I_{i,1} \,\|\, 0^c),\ S_{i+1,2} \leftarrow \pi(S_{i,1} \oplus I_{i,2} \,\|\, 0^c),$$
$$I_{i+1,3} \leftarrow S^{(r)}_{i,1} \oplus I_{i,1} \oplus S^{(r)}_{i,2},\ I_{i+1,4} \leftarrow S^{(r)}_{i,1} \oplus I_{i,2} \oplus S^{(r)}_{i,2}.$$

This ensures:

$$S_{i+1,1} = \pi(S_{i,2} \oplus I_{i,3} \,\|\, 0^c),\ S_{i+1,2} = \pi(S_{i,2} \oplus I_{i,4} \,\|\, 0^c),$$

maintaining a collision in the lower c bits reachable from both $S_{i,1}$ and $S_{i,2}$.

4. $\mathcal{A}_1$ sends $(S_{\lambda+1,1}, S_{\lambda+1,2})$ to $\mathcal{A}_2$ and constructs the entropy input $I = I_1 \,\|\, I_2 \,\|\, \cdots \,\|\, I_\lambda$ as follows:
   (a) Uniformly selects $I_1 \leftarrow_\$ \{I_{1,1}, I_{1,2}\}$.
   (b) If $I_1 = I_{1,1}$, then $I_2 \leftarrow_\$ \{I_{2,1}, I_{2,2}\}$. Otherwise, $I_2 \leftarrow_\$ \{I_{2,3}, I_{2,4}\}$.
   (c) If $I_2 = I_{2,1}$ or $I_{2,3}$, then $I_3 \leftarrow_\$ \{I_{3,1}, I_{3,2}\}$. Otherwise, $I_3 \leftarrow_\$ \{I_{3,3}, I_{3,4}\}$.
   (d) For $i = 4$ to λ, choose I_i based on the previous selection, ensuring consistent paths in the constructed collision chain similar to the above case.

   Since each I_i is selected uniformly from a two-element set, the probability of choosing any specific I is $1/2^\lambda$, satisfying the λ-legitimacy condition.
5. $\mathcal{A}_2$ queries SET(0^n); $\mathcal{A}_1$ queries REF(I, λ); $\mathcal{A}_2$ queries ROR(r) and get its return value y.
   – If $y \in \{S^{(r)}_{\lambda+1,1}, S^{(r)}_{\lambda+1,2}\}$, $\mathcal{A}_2$ outputs 1,
   – Otherwise, $\mathcal{A}_2$ outputs 0.

By the construction of I, the DRBG state will be either $S_{\lambda+1,1}$ or $S_{\lambda+1,2}$ after querying REF(I, λ).

– Real World: ROR(r) outputs $S^{(r)}_{\lambda+1,1}$ or $S^{(r)}_{\lambda+1,2}$ with probability 1,
– Ideal World: The output is uniformly random, with a probability of $2/2^r$ of matching $S^{(r)}_{\lambda+1,1}$ or $S^{(r)}_{\lambda+1,2}$.

Thus, the advantage of $\mathcal{A}$ is non-negligible, achieved with $O(\lambda \cdot 2^{c/2})$ queries.

Attack 2: This attack is an application of the attack given in [6] and its complexity is $O(2^{\lambda/2})$: when $\lambda < c$, a λ-legitimate adversary $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$ wins the robustness game with high probability by executing the following steps.

1. $\mathcal{A}_1$ chooses entropy inputs uniformly at random from set $\mathcal{T}$ where $|\mathcal{T}| = 2^\lambda$. Note that $\mathcal{A}$ is still λ-legitimate.
2. For any $S^* \in \mathcal{S}$, and distinct $I_1 \cdots, I_\alpha \in \mathcal{T}$, $\mathcal{A}_2$ simulates refresh using S^* and I_i and repeatedly querying π, and obtains S_i, $1 \le i \le \alpha$, as the responses.
3. $\mathcal{A}_2$ simulates next using S_i and repeatedly querying π for $1 \le i \le \alpha$, and obtains $3n$-bit outputs. The outputs are recorded in $\mathcal{X}$.
4. $\mathcal{A}_2$ makes a query SET(S^*); $\mathcal{A}_1$ chooses $I_1 \,\|\, \cdots \,\|\, I_{\mathsf{len}} \xleftarrow{r} I \in \mathcal{T}$ and makes queries REF$[\pi](I_1)$, …, REF$[\pi](I_{\mathsf{len}})$. Then $\mathcal{A}_2$ makes a query ROR$[\pi](3n)$ to obtain random bits. This step is repeated β times, and the random bits are recorded in $\mathcal{Y}$.

5. If $\mathcal{X} \cap \mathcal{Y} = \emptyset$, $\mathcal{A}_2$ outputs 0. Otherwise, $\mathcal{A}_2$ outputs 1.

To make an intersection, in the real world, it is sufficient to make the collision between entropy input and simulated entropy input. However, in the ideal world, the output bits are generated uniformly at random. Therefore we have

$$\Pr\left[1 \leftarrow \mathcal{A} \mid b = 0\right] = \frac{\alpha\beta}{2^{3n}},$$
$$\Pr\left[1 \leftarrow \mathcal{A} \mid b = 1\right] = 1 - \left(1 - \frac{\alpha}{2^{\lambda}}\right)^{\beta} \geq \frac{\alpha\beta}{2^{\lambda}} - \frac{(\alpha\beta)^2}{2^{2\lambda+1}}.$$

Therefore, if $\alpha = \beta = 2^{\frac{\lambda}{2}}$, the advantage of $\mathcal{A}$ is non-negligible.

Acknowledgements. Jooyoung Lee was supported by the Institute of Information & Communications Technology Planning & Evaluation(IITP) grant funded by the Korea government(MSIT) (No.RS-2024-00399491, Development of Privacy-Preserving Multiparty Computation Techniques for Secure Multiparty Data Integration). We gratefully acknowledge the reviewers' insightful comments, which have enhanced this manuscript.

Appendix

A Description of M-EXT Game.

The M-EXT game was originally proposed in [6]. Since it was described in the context of Linux-DRBG's syntax, we present the M-EXT game in this section using the syntax of a general DRBG to improve clarity and understanding. The M-EXT game is defined as follows, using the oracles defined in Algorithm 7.

M-EXT Game

1. INIT is executed.
2. $\mathcal{A}$ makes queries to M-EXT, and P (and P^{-1}). A query to M-EXT is made as follows.
   (a) $\mathcal{A}_2$ chooses an initial state s_0, and then returns it to $\mathcal{A}_1$ through S_{all}.
   (b) $\mathcal{A}_1$ chooses entropy input I, makes queries M-EXT(s_0, I) and then passes the responses to $\mathcal{A}_2$ through S_{all}.
3. $\mathcal{A}_2$ outputs $b' \in \{0,1\}$.
4. If $b' = b$, then $\mathcal{A}$ wins, and $\mathcal{A}$ loses otherwise.

Algorithm 7 Oracles for M-EXT game

Procedure INIT()
$b \leftarrow_{\$} \{0,1\}$
$P \leftarrow_{\$} \mathcal{P}$

Procedure M-EXT(s_0, I)
if $b = 0$ **then**
  $s \leftarrow_{\$} \{0,1\}^n$
else
  $(I_1, \ldots, I_\ell) \xleftarrow{r} I$
  for $i \leftarrow 1$ to ℓ **do**
    $s_i \leftarrow \mathsf{refresh}[P](s_{i-1}, I_i)$
  $s \leftarrow s_\ell$
return (s_0, s)

Procedure $P(x)$
$y = P(x)$
return y

Procedure $P^{-1}(y)$ *//If available*
$x = P^{-1}(y)$
return x

References

1. B. Barak and S. Halevi. A Model and Architecture for Pseudo-Random Generation with Applications to /Dev/Random. In *Proceedings of the 12th ACM Conference on Computer and Communications Security*, CCS '05, page 203–212, New York, NY, USA, 2005. Association for Computing Machinery.
2. G. Bertoni, J. Daemen, S. Hoffert, M. Peeters, G. Van Assche, R. Van Keer, and B. Viguier. TurboSHAKE. *Cryptology ePrint Archive*, 2023.
3. G. Bertoni, J. Daemen, M. Peeters, and G. Van Assche. Sponge-based pseudo-random number generators. In *Cryptographic Hardware and Embedded Systems, CHES 2010: 12th International Workshop, Santa Barbara, USA, August 17-20, 2010. Proceedings 12*, pages 33–47. Springer, 2010.
4. G. Bertoni, J. Daemen, M. Peeters, and G. Van Assche. Keccak. In *Annual international conference on the theory and applications of cryptographic techniques*, pages 313–314. Springer, 2013.
5. G. Bertoni, J. Daemen, M. Peeters, G. Van Assche, R. Van Keer, and B. Viguier. Kangaroo Twelve: Fast Hashing Based on Keccak-p KECCAK-p. In *Applied Cryptography and Network Security: 16th International Conference, ACNS 2018, Leuven, Belgium, July 2-4, 2018, Proceedings 16*, pages 400–418. Springer, 2018.
6. W. Chung, H. Kim, J. Lee, and Y. Lee. Provable Security of Linux-DRBG in the Seedless Robustness Model. In *ASIACRYPT 2024, to appear*.
7. W. Chung, H. Kim, J. Lee, and Y. Lee. Security analysis of the ISO standard OFB-DRBG. *Designs, Codes and Cryptography*, pages 1–18, 2024.
8. S. Coretti, Y. Dodis, H. Karthikeyan, and S. Tessaro. Seedless fruit is the sweetest: Random number generation, revisited. In *Annual International Cryptology Conference*, pages 205–234. Springer, 2019.
9. J. Daemen, S. Hoffert, S. Mella, and G. V. Assche. SHAKE modes of operation. 2023.
10. Y. Dodis, D. Pointcheval, S. Ruhault, D. Vergniaud, and D. Wichs. Security Analysis of Pseudo-Random Number Generators with Input: /Dev/Random is Not Robust. CCS '13, page 647–658, New York, NY, USA, 2013. Association for Computing Machinery.
11. Y. Dodis, A. Shamir, N. Stephens-Davidowitz, and D. Wichs. How to eat your entropy and have it too: Optimal recovery strategies for compromised RNGs. *Algorithmica*, 79:1196–1232, 2017.
12. M. J. Dworkin. SHA-3 standard: Permutation-based hash and extendable-output functions. 2015.
13. N. Ferguson and B. Schneier. *Practical cryptography*, volume 141. Wiley New York, 2003.
14. S. L. Garfinkel and P. Leclerc. Randomness concerns when deploying differential privacy. In *Proceedings of the 19th Workshop on Privacy in the Electronic Society*, pages 73–86, 2020.
15. P. Gaži and S. Tessaro. Provably robust sponge-based PRNGs and KDFs. In *Advances in Cryptology–EUROCRYPT 2016: 35th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Vienna, Austria, May 8-12, 2016, Proceedings, Part I 35*, pages 87–116. Springer, 2016.
16. V. T. Hoang and Y. Shen. Security Analysis of NIST CTR-DRBG. In *Advances in Cryptology–CRYPTO 2020: 40th Annual International Cryptology Conference, CRYPTO 2020, Santa Barbara, CA, USA, August 17–21, 2020, Proceedings, Part I*, pages 218–247. Springer, 2020.

17. D. Hutchinson. A robust and sponge-like PRNG with improved efficiency. In *Selected Areas in Cryptography–SAC 2016: 23rd International Conference, St. John's, NL, Canada, August 10-12, 2016, Revised Selected Papers 23*, pages 381–398. Springer, 2017.
18. S. Ruhault. SoK: Security models for pseudo-random number generators. *IACR Transactions on Symmetric Cryptology*, pages 506–544, 2017.
19. T. Shrimpton and R. S. Terashima. A provable-security analysis of Intel's secure key RNG. In *Advances in Cryptology–EUROCRYPT 2015: 34th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Sofia, Bulgaria, April 26-30, 2015, Proceedings, Part I*, pages 77–100. Springer, 2015.
20. M. Sönmez Turan, K. McKay, D. Chang, J. Kang, and J. Kelsey. Ascon-Based Lightweight Cryptography Standards for Constrained Devices: Authenticated Encryption, Hash, and Extendable Output Functions. Technical report, National Institute of Standards and Technology, 2024.
21. J. Woodage and D. Shumow. An Analysis of NIST SP 800-90A. 11477:151–180, 2019.